CHAPTER 17

# The Standard Template Library

**TOPICS**


17.1 Introduction to the Standard Template Library
17.2 STL Container and Iterator Fundamentals
17.3 The `vector` Class
17.4 The `map`, `multimap`, and `unordered_map` Classes
17.5 The `set`, `multiset`, and `unordered_set` Classes
17.6 Algorithms
17.7 Introduction to Function Objects and Lambda Expressions


## 17.1 Introduction to the Standard Template Library

**CONCEPT: The Standard Template Library is an extensive collection of templates for useful data structures and algorithms.**

In addition to its runtime library, which you have used throughout this book, C++ also provides an extensive library of templates. The *Standard Template Library* (or *STL*) contains numerous generic templates for classes and functions. Most of the templates in the STL can be grouped into the following categories:

- **Containers:** Class templates for objects that store and organize data
- **Iterators:** Class templates for objects that behave like pointers, and are used to access the individual data elements in a container
- **Algorithms:** Function templates that perform various operations on elements of containers

This chapter introduces many of the STL's class and function templates.

## 17.2 STL Container and Iterator Fundamentals

**CONCEPT: A container is an object that holds a collection of values or other objects. An iterator is an object that is used to iterate over the items in a collection, providing access to them.**

## Containers

There are two types of container classes in the STL: *sequence* and *associative*. A sequence container stores its data sequentially in memory, in a fashion similar to an array. An associative container stores its data in a nonsequential way that makes it faster to locate elements.

The sequence containers currently provided in the STL are listed in Table 17-1. (Note that the `deque` container will be discussed in Chapter 19, and the `list` and `forward_list` containers will be discussed in Chapter 18.)

**Table 17-1** Sequence Containers

| Container Class | Description |
|---|---|
| `array` | A fixed-size container that is similar to an array |
| `deque` | A double-ended queue. Like a `vector`, but designed so that values can be quickly added to or removed from the front and back. (This container will be discussed in Chapter 19.) |
| `forward_list` | A singly linked list of data elements. Values may be inserted to or removed from any position. (This container will be discussed in Chapter 18.) |
| `list` | A doubly linked list of data elements. Values may be inserted to or removed from any position. (This container will be discussed in Chapter 18.) |
| `vector` | A container that works like an expandable array. Values may be added to or removed from a `vector`. The `vector` automatically adjusts its size to accommodate the number of elements it contains. |

The associative containers currently provided in the STL are listed in Table 17-2.

**Table 17-2** Associative Containers

| Container Class | Description |
|---|---|
| `set` | Stores a set of unique values that are sorted. No duplicates are allowed. |
| `multiset` | Stores a set of unique values that are sorted. Duplicates are allowed. |
| `map` | Maps a set of keys to data elements. Only one key per data element is allowed. Duplicates are not allowed. The elements are sorted in order of their keys. |
| `multimap` | Maps a set of keys to data elements. Many keys per data element are allowed. Duplicates are allowed. The elements are sorted in order of their keys. |
| `unordered_set` | Like a `set`, except that the elements are not sorted |
| `unordered_multiset` | Like a `multiset`, except that the elements are not sorted |
| `unordered_map` | Like a `map`, except that the elements are not sorted |
| `unordered_multimap` | Like a `multimap`, except that the elements are not sorted |

In addition to the classes listed in Tables 17-1 and 17-2, the STL also provides the three container adapter classes listed in Table 17-3. A container *adapter class* is not itself a container,

but a class that adapts one of the other containers to a specific use. Note we will discuss these classes in Chapter 19.

**Table 17-3** Container Adapter Classes

| Container Adapter Class | Description |
|---|---|
| `stack` | An adapter class that stores elements in a `deque` (by default). A `stack` is a last-in, first-out (LIFO) container. When you retrieve an element from a `stack`, the `stack` always gives you the last element that was inserted. (This class will be discussed in Chapter 19.) |
| `queue` | An adapter class that stores elements in a `deque` (by default). A `queue` is a first-in, first-out (FIFO) container. When you retrieve an element from a `stack`, the `stack` always gives you the first, or earliest, element that was inserted. (This class will be discussed in Chapter 19.) |
| `priority_queue` | An adapter class that stores elements in a `vector` (by default). A data structure in which the element that you retrieve is always the element with the greatest value. (This class will be discussed in Chapter 19.) |

The container and container adapter classes are declared in various STL header files. Table 17-4 lists the necessary header files that you will need to include, depending on the containers you intend to use.

**Table 17-4** Header Files

| Header File | Classes |
|---|---|
| `<array>` | `array` |
| `<deque>` | `deque` |
| `<forward_list>` | `forward_list` |
| `<list>` | `list` |
| `<map>` | `map, multimap` |
| `<queue>` | `queue, priority_queue` |
| `<set>` | `set, multiset` |
| `<stack>` | `stack` |
| `<unordered_map>` | `unordered_map, unordered_multimap` |
| `<unordered_set>` | `unordered_set, unordered_multiset` |
| `<vector>` | `vector` |

## Introduction to the `array` Class



Perhaps the simplest container in the STL is the `array` class, which was introduced in C++ 11. An `array` object works very much like a regular array. It is a fixed-size container that holds elements of the same data type. In fact, an `array` object uses a traditional array, internally, to store its elements. An advantage `array` objects have over regular arrays is that `array` objects have a `size()` member function that returns the number of elements contained in the object.


VideoNote
**The array Container**

When you define an `array` object, you must specify the data type of its elements, and the number of elements the object will store. Here is an example:

```
array<int, 5> numbers;
```

This statement defines an `array` object named `numbers`. Inside the angled brackets <>, the data type `int` is the type of each element, and the value 5 is the number of elements the object will have. You can provide an initialization list to initialize an `array`, as shown here:

```
array<int, 5> numbers = {1, 2, 3, 4, 5};
```

Here is an example of defining an `array` to hold four strings:

```
array<string, 4> names = {"Jamie", "Ashley", "Doug", "Claire"};
```

The `array` class overloads the `[]` operator, giving you the ability to access elements using a subscript, just as you would with a regular array. Program 17-1 gives a simple demonstration of creating and initializing an `array` object, and displaying its elements.

**Program 17-1**

```
 1  #include <iostream>
 2  #include <string>
 3  #include <array>
 4  using namespace std;
 5
 6  int main()
 7  {
 8      const int SIZE = 4;
 9
10      // Store some names in an array object.
11      array<string, SIZE> names = {"Jamie", "Ashley", "Doug", "Claire"};
12
13      // Display the names.
14      cout << "Here are the names:\n";
15      for (int index = 0; index < names.size(); index++)
16         cout << names[index] << endl;
17
18      return 0;
19  }
```

**Program Output**

```
Here are the names:
Jamie
Ashley
Doug
Claire
```

Take a closer look at the following lines of code in Program 17-1:

- The statement in line 11 defines an `array` object named `names`, initialized with four strings.
- The `for` loop in lines 15 and 16 displays all of the strings. Notice the `array` object's `size()` member function is used in the loop's test expression.
- In line 16, the `[]` operator is used to access the object's elements.

The `for` loop in lines 15 and 16 of Program 17-1 is useful to demonstrate the use of the `array` class's `size()` member function, but we could easily rewrite it as a range-based `for` loop.

```
for (auto element : names)
   cout << element << endl;
```



**WARNING!** The `array` class's `[]` operator does not perform bounds checking. As with regular arrays, you must be careful not to use a subscript that is out of bounds with an `array` object.

As you can see, the `array` class is an object-oriented alternative to a traditional array in C++. The `array` class also provides several member functions that give additional capabilities. Table 17-5 lists the `array` class's member functions.

**Table 17-5** The `array` Member Functions

| Member Function | Description |
|---|---|
| `at(`*`index`*`)` | Returns a reference to the element located at the specified index. If the specified index is out of bounds, the `at` function throws an `out_of_bounds` exception. |
| `back()` | Returns a reference to the last element in the container. |
| `begin()` | Returns an iterator to the first element in the container. |
| `cbegin()` | Returns a `const_iterator` to the first element in the container. |
| `cend()` | Returns a `const_iterator` pointing to the end of the container. |
| `crbegin()` | Returns a `const_reverse_iterator` pointing to the last element in the container. |
| `crend()` | Returns a `const_reverse_iterator` pointing to the first element in the container. |
| `data()` | Returns a pointer to the first element in the container. The `array` class uses a traditional array to store its data, so the pointer returned from the `data()` function points to the first element in the underlying array. |
| `empty()` | Returns `true` if the container is empty, or `false` otherwise. |
| `end()` | Returns an iterator pointing to the end of the container. |
| `fill(`*`value`*`)` | Sets the value of each element to *`value`*. |
| `front()` | Returns a reference to the first element in the container. |
| `max_size()` | Returns the number of elements in the container (the same value returned by the `size()` member function). |
| `rbegin()` | Returns a `reverse_iterator` pointing to the last element in the container. |
| `rend()` | Returns a `reverse_iterator` pointing to the first element in the container. |
| `size()` | Returns the number of elements in the container. |
| `swap(`*`second`*`)` | The *`second`* argument must be an `array` object of the same type and size as the calling object. The function swaps the contents of the calling object and the *`second`* object. |

Notice many of the member functions listed in Table 17-5 return iterators. Iterators are a fundamental part of the STL, so it is important to have an understanding of them.


VideoNote
**Iterators**

## Iterators

Iterators are objects that work like pointers, and are used to access data stored in containers. As the name implies, you can use an iterator to iterate over the contents of a container. There are currently five categories of iterators, as described in Table 17-6.[1]

**Table 17-6** Categories of Iterators

| Iterator Category | Description |
|---|---|
| Forward | Can only move forward in a container (uses the `++` operator). |
| Bidirectional | Can move forward or backward in a container (uses the `++` and `--` operators). |
| Random access | Can move forward and backward, and can jump to a specific data element in a container. |
| Input | Can be used with an input stream to read data from an input device or a file. |
| Output | Can be used with an output stream to write data to an output device or a file. |

Iterators are associated with containers. The type of container you have determines the category of iterator you use. For example:

- The `array`, `vector`, and `deque` containers use random-access iterators.
- The `list`, `set`, `multiset`, `map`, and `multimap` containers use bidirectional iterators.
- The `forward_list`, `unordered_map`, `unordered_multimap`, `unordered_set`, and `unordered_multiset` containers use forward iterators.

Iterators are like pointers in the following ways:

- An iterator can point to an element in a container.
- You can use the `*` operator to dereference an iterator, getting the element that it points to.
- When an iterator points to an object, you can use the `->` operator to access the object's members.
- You can use the `=` operator to assign an iterator to an element.
- You can use the `==` and `!=` operators to compare two iterators.
- You can use the `++` operator to increment an iterator, thus moving it to the next element in the container. You can also use the `+` operator to add an integer to an iterator, thus moving it forward in the container by a number of elements.
- If you are using a bidirectional or random-access iterator, you can use the `--` operator to decrement the iterator, thus moving it to the previous element in the container. You can also use the `-` operator to subtract an integer from an iterator, thus moving it backward in the container by a number of elements.

### Defining an Iterator

Before you can define an iterator, you have to know the type of container with which you want to use the iterator. The general format of an iterator definition is:

```
containerType::iterator  iteratorName;
```

[1] The C++17 standard will introduce a sixth category, the contiguous iterator. A contiguous iterator is a random-access iterator that points to elements known to be stored in contiguous memory locations.

In the general format, *containerType* is the STL container type, and *iteratorName* is the name of the iterator variable that you are defining. For example, suppose we have defined an `array` object, as follows:

```
array<string, 3> names = {"Sarah", "William", "Alfredo"};
```

We can define an iterator that will work with the `array` object with the following statement:

```
array<string, 3>::iterator it;
```

This statement defines an iterator named `it`, that can be used with an `array<string, 3>` object. The compiler automatically chooses the right type of iterator (in this case, a random-access iterator).

### Getting an Iterator from a Container Object

All of the container classes in the STL provide a `begin()` member function and an `end()` member function. The `begin()` member function is straightforward: It returns an iterator pointing to the first element in a container. For example, the following code snippet does the following:

1. It defines an `array` object initialized with three strings.
2. It defines an iterator that can be used with the `array` object.
3. It makes the iterator point to the first element in the `array` object.
4. It uses the `*` operator to dereference the iterator, and displays the element to which the iterator points.

```
// Define an array object. (1)
array<string, 3> names = {"Sarah", "William", "Alfredo"};

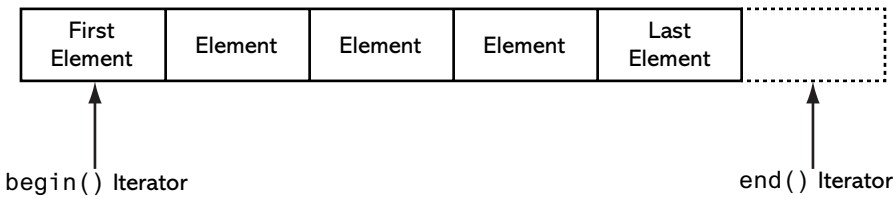
// Define an iterator for the array object. (2)
array<string, 3>::iterator it;

// Make the iterator point to the array object's first element. (3)
it = names.begin();

// Display the element that the iterator points to. (4)
cout << *it << endl;
```

The `end()` member function returns an iterator pointing to the position *after* the last element. This means it returns an iterator that does not point to an element, but points to the end of the container. This is illustrated in Figure 17-1.

**Figure 17-1** Iterators returned from the `begin()` and `end()` member functions




You typically use the `end()` member function to know when you have reached the end of a container in algorithms that iterate over a range of elements. For example, the following

code snippet shows an iterator being used with a `while` loop to display the contents of an `array` object:

```
// Define an array object.
array<string, 3> names = {"Sarah", "William", "Alfredo"};

// Define an iterator for the array object.
array<string, 3>::iterator it;

// Make the iterator point to the array object's first element.
it = names.begin();

// Display the array object's contents.
while (it != names.end())
{
   cout << *it << endl;
   it++;
}
```

The code previously shown could be shortened somewhat by using a `for` loop instead of a `while` loop. Program 17-2 shows an example.

**Program 17-2**

```
1  #include <iostream>
2  #include <string>
3  #include <array>
4  using namespace std;
5
6  int main()
7  {
8       const int SIZE = 3;
9
10      // Store some names in an array object.
11      array<string, SIZE> names = {"Sarah", "William", "Alfredo"};
12
13      // Create an iterator for the array object.
14      array<string, SIZE>::iterator it;
15
16      // Display the names.
17      cout << "Here are the names:\n";
18      for (it = names.begin(); it != names.end(); it++)
19           cout << *it << endl;
20
21      return 0;
22 }
```

**Program Output**

```
Here are the names:
Sarah
William
Alfredo
```

### Using auto to Define an Iterator

Quite often, you can use auto to simplify the definition of an iterator. For example, any time you are initializing an iterator with the value that is returned from a member function, such as begin(), you can use auto in the iterator's declaration. For example, look at the following code:

```
array<string, 3> names = {"Sarah", "William", "Alfredo"};
array<string, 3>::iterator it = names.begin();
```

The second statement defines an iterator named it, and calls the names.begin() member function to initialize it. This statement can be rewritten, as shown in the following code:

```
array<string, 3> names = {"Sarah", "William", "Alfredo"};
auto it = names.begin();
```

The auto declaration shown in the second statement works because the names.begin() function returns an object of the array<string, 3>::iterator type. So, the compiler can determine that it should be an object of the array<string, 3>::iterator type. This is especially useful in for loops that use iterators to step through the elements of a container. For example, look at Program 17-3. It is similar to Program 17-2, but in this version, the iterator is auto declared in the initialization expression of the for loop (in line 15).

**Program 17-3**

```
1  #include <iostream>
2  #include <string>
3  #include <array>
4  using namespace std;
5
6  int main()
7  {
8      const int SIZE = 4;
9
10     // Store some names in an array object.
11     array<string, SIZE> names = {"Jamie", "Ashley", "Doug", "Claire"};
12
13     // Display the names.
14     cout << "Here are the names:\n";
15     for (auto it = names.begin(); it != names.end(); it++)
16        cout << *it << endl;
17
18     return 0;
19 }
```

**Program Output**

```
Here are the names:
Sarah
William
Alfredo
```

### Mutable Iterators and const_iterators

An iterator of the iterator type gives you read/write access to the element to which the iterator points. This is commonly known as a *mutable iterator*. The following code snippet shows an example:

```
// Define an array object.
array<int, 5> numbers = {1, 2, 3, 4, 5};

// Define an iterator for the array object.
array<int, 5>::iterator it;

// Make the iterator point to the array object's first element.
it = numbers.begin();

// Use the iterator to change the element.
*it = 99;
```

In this code snippet, the numbers object initially contains the values 1, 2, 3, 4, and 5. The last statement dereferences the it iterator (which points to the first element) and changes the element's value to 99. After the last statement executes, the numbers object contains the values 99, 2, 3, 4, and 5.

If you want to make sure that nothing changes the contents of a container, you can use the const modifier with the container definition to make the container's contents constant. Here is an example:

```
const array<string, 3> names = {"Sarah", "William", "Alfredo"};
```

This statement defines names as a const array object. As a result, the elements in the container are constant, and cannot be modified. (Attempting to modify the container's contents will result in an error at compile time.) If we need an iterator to work with the object, we must use a const_iterator, which provides read-only access to any element to which it points. The following statement shows an example of how to define a const_iterator:

```
array<string, 3>::const_iterator it;
```

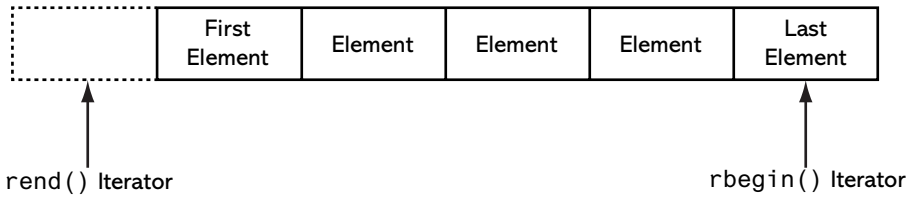
All of the container classes in the STL provide a cbegin() member function and a cend() member function. The cbegin() member function returns a const_iterator pointing to the first element in a container. The cend() member function returns a const_iterator pointing to the end of the container. When working with const_iterators, simply use the container class's cbegin() and cend() member functions instead of the begin() and end() member functions.

### Reverse Iterators

A *reverse iterator* is a bidirectional or random-access iterator that works in reverse, allowing you to iterate backward over the elements in a container. When using a reverse iterator, the last element in a container is considered the first element, and the first element is considered the last element. The ++ operator moves a reverse iterator backward, and the -- operator moves a reverse iterator forward.

The following STL containers support the use of reverse iterators: array, deque, list, map, multimap, multiset, set, and vector. All of these classes provide an rbegin() member function and an rend() member function. The rbegin() member function returns a reverse iterator pointing to the last element in a container, and the rend() member function returns an iterator pointing to the position *before* the first element. This is illustrated in Figure 17-2.

**Figure 17-2** Iterators returned by the `rbegin()` and `rend()` member functions




To create a reverse iterator, you define it as `reverse_iterator`. The following code snippet shows an example of using a reverse iterator to display the contents of an `array` object in reverse order:

```
// Define an array object.
array<int, 5> numbers = {1, 2, 3, 4, 5};

// Define a reverse iterator for the array object.
array<int, 5>::reverse_iterator it;

// Display the elements in reverse order.
for (it = numbers.rbegin(); it != numbers.rend(); it++)
   cout << *it << endl;
```

Iterators that are declared as `reverse_iterator` are mutable, meaning they give you read/write access to the element being pointed to. If you need a reverse iterator to work with a `const` container, you must use a `const_reverse_iterator`, which provides read-only access to any element that it points to. The following code shows an example of how to create a `const_reverse_iterator`:

```
const array<string, 3> names = {"Sarah", "William", "Alfredo"};
array<string, 3>::const_reverse_iterator it;
```

Each of the STL container classes that supports reverse iterators provides a `crbegin()` member function and a `crend()` member function. The `crbegin()` member function returns a `const_reverse_iterator` pointing to the last element in the container. The `crend()` member function returns a `const_reverse_iterator` pointing to the position before the first element in the container.



## Checkpoint

17.1 What two types of containers does the STL provide?

17.2 What is a container adapter class?

17.3 What is an iterator?

17.4 Suppose you are writing a program that uses the `array`, `multimap`, and `vector` classes. What header files must you `#include` in the program, in order to use these classes?

17.5 What is the difference between a bidirectional iterator and a random-access iterator?

17.6 What does the `++` operator do when applied to an iterator?

17.7 What does a container's `begin()` and `end()` member functions return?

17.8 What is the difference between a mutable iterator and a `const_iterator`?

17.9 What is a reverse iterator?

17.10 What does a container's `rbegin()` and `rend()` member functions return?

## 17.3 The vector Class

**CONCEPT:** **A vector is a sequence container that works like an array, but is dynamic in size.**



VideoNote
**The vector Container**

Chapter 7 introduces the `vector` class and discusses several of the class's member functions. In this section, we will take a closer look at the class, and discuss how to use iterators to access a `vector` container's elements.

Recall from our previous discussions that a `vector` stores a sequence of elements, like an array. In fact, a `vector` uses an array internally to store its elements. A `vector`, however, offers many advantages over an array:

- You do not have to declare the size of a `vector` when you define it.
- A `vector` is dynamic in size. You can add elements to it, or delete elements from it at runtime. The `vector` automatically adjusts its size according to the number of elements it contains.
- A `vector` can report the number of elements it contains.

To use the `vector` class, you need to `#include` the `<vector>` header file in your program. Then, you can define a `vector` object using one of the four `vector` constructors. Table 17-7 shows the general format of a `vector` definition statement using each constructor. Table 17-8 lists most of the `vector` class's member functions.

**Table 17-7** `vector` Definition Statements

| | |
|---|---|
| **Default Constructor** | `vector<dataType> name;`<br>Creates an empty `vector` object. In the general format, *`dataType`* is the data type of each element, and *`name`* is the name of the `vector`. |
| **Fill Constructor** | `vector<dataType> name(size);`<br>Creates a `vector` object of a specified size. In the general format, *`dataType`* is the data type of each element, and *`name`* is the name of the `vector`. The *`size`* argument is an unsigned integer that specifies the number of elements that the `vector` should initially have. If the elements are objects, they are initialized via their default constructors. Otherwise, the elements are initialized with the value 0. |
| **Fill Constructor** | `vector<dataType> name(size, value);`<br>Creates a `vector` object of a specified size, where each element is initially given a specified value. In the general format, *`dataType`* is the data type of each element, and *`name`* is the name of the `vector`. The *`size`* argument is an unsigned integer that specifies the number of elements that the `vector` should initially have, and the *`value`* argument is the value with which to fill each element. |

**Table 17-7** *(continued)*

| | |
|---|---|
| **Range Constructor** | `vector<dataType> name(iterator1, iterator2);`<br>Creates a `vector` object that initially contains a range of values specified by two iterators. In the general format, *`dataType`* is the data type of each element, and *`name`* is the name of the `vector`. The *`iterator1`* and *`iterator2`* arguments mark the beginning and end of a range of values that will be stored in the `vector`. |
| **Copy Constructor** | `vector<dataType> name(vector2);`<br>Creates a `vector` object that is a copy of another `vector` or object. In the general format, *`dataType`* is the data type of each element, *`name`* is the name of the `vector`, and *`vector2`* is the `vector` to copy. |

**Table 17-8** The `vector` Member Functions

| Member Function | Description |
|---|---|
| `assign(size, value)` | Assigns a new set of elements to the container, replacing its existing elements. The *`size`* argument is an unsigned integer that specifies the number of elements that the `vector` should have, and the *`value`* argument is the value with which to fill each element. After the function executes, the container will have *`size`* elements, each set to *`value`*. |
| `assign(iterator1, iterator2)` | Assigns a new set of elements to the container, replacing its existing contents with a range of values specified by iterators. The *`iterator1`* and *`iterator2`* arguments mark the beginning and end of a range of values that will be stored in the container. |
| `at(index)` | The *`index`* argument is an unsigned integer. The function returns a reference to the element located at the specified index. (The first element is at index 0, the second element is at index 1, and so on.) If the specified index is out of bounds, the `at` function throws an `out_of_bounds` exception. |
| `back()` | Returns a reference to the last element in the container. |
| `begin()` | Returns an iterator to the first element in the container. |
| `capacity()` | Returns the number of elements that the container's underlying array can hold without reallocating the array. |
| `cbegin()` | Returns a `const_iterator` to the first element in the container. |
| `cend()` | Returns a `const_iterator` pointing to the end of the container. |
| `clear()` | Erases all of the elements in the container. |
| `crbegin()` | Returns a `const_reverse_iterator` pointing to the last element in the container. |
| `crend()` | Returns a `const_reverse_iterator` pointing to the first element in the container. |
| `data()` | Returns a pointer to the first element in the container. The `vector` class uses a traditional array to store its data, so the pointer returned from the `data()` function points to the first element in the underlying array. |

*(table continues)*

**Table 17-8** *(continued)*

| Member Function | Description |
|---|---|
| `emplace(it, value)` | Constructs a new element with *value* as its value. The *it* argument is an iterator pointing to an existing element in the container. The new element will be inserted before the one pointed to by *it*. |
| `emplace_back(value)` | Constructs a new element containing the specified *value* at the end of the container. |
| `empty()` | Returns `true` if the container is empty, or `false` otherwise. |
| `end()` | Returns an iterator pointing to the end of the container. |
| `erase(it)` | Erases the element pointed to by the iterator *it*. This function returns an iterator pointing to the element that follows the removed element (or the end of the container, if the removed element was the last one). |
| `erase(iterator1, iterator2)` | Erases a range of elements. The *iterator1* and *iterator2* arguments mark the beginning and end of a range of values that will be erased. This function returns an iterator pointing to the element that follows the removed elements (or the end of the container, if the last element was erased). |
| `front()` | Returns a reference to the first element in the container. |
| `insert(it, value)` | Inserts a new element with *value* as its value. The *it* argument is an iterator pointing to an existing element in the container. The new element will be inserted before the one pointed to by *it*. The function returns an iterator pointing to the newly inserted element. |
| `insert(it, n, value)` | Inserts *n* new elements with *value* as their value. The *it* argument is an iterator pointing to an existing element in the container, and *n* is an unsigned integer. The new elements will be inserted before the one pointed to by *it*. The function returns an iterator pointing to the first element of the newly inserted elements. |
| `insert(iterator1, iterator2, iterator3)` | Inserts a range of new elements. The *iterator1* argument points to an existing element in the container. The range of new elements will be inserted before the element pointed to by *iterator1*. The *iterator2* and *iterator3* arguments mark the beginning and end of a range of values that will be inserted. (The element pointed to by *iterator3* will not be included in the range.) The function returns an iterator pointing to the first element of the newly inserted range. |
| `max_size()` | Returns the theoretical maximum size of the container. |
| `pop_back()` | Removes the last element of the container. |
| `push_back(value)` | Adds a new element containing *value* to the end of the container. |
| `rbegin()` | Returns a `reverse_iterator` pointing to the last element in the container. |
| `rend()` | Returns a `reverse_iterator` pointing to the first element in the container. |
| `resize(n)` | The *n* argument is an unsigned integer. This function resizes the container so it has *n* elements. If the current size of the container is larger than *n*, then the container is reduced in size so it keeps only the first *n* elements. If the current size of the container is smaller than *n*, then the container is increased in size so it has *n* elements. |

**Table 17-8** *(continued)*

| Member Function | Description |
|---|---|
| `resize(n, value)` | Resizes the container so it has *n* elements (the *n* argument is an unsigned integer). If the current size of the container is larger than *n*, then the container is reduced in size so it keeps only the first *n* elements. If the current size of the container is smaller than *n*, then the container is increased in size so that it has *n* elements, and each of the new elements is initialized with *value*. |
| `shrink_to_fit()` | Requests that the `vector` be resized so its capacity is the same as its size. |
| `size()` | Returns the number of elements in the container. |
| `swap(second)` | The *second* argument must be a `vector` object of the same type as the calling object. The function swaps the contents of the calling object and the *second* object. |

## Review of Basic `vector` Operations

Here is an example of how you would use the default constructor to define a `vector` container to hold integers:

```
vector<int> numbers;
```

This statement defines an empty `vector` container named `numbers`. Inside the angled brackets `<>`, the data type `int` is the type of each element. If you are using a compiler that is compliant with C++11 or later, you can use an initialization list to initialize the `vector`, as shown here:

```
vector<int> numbers = {1, 2, 3, 4, 5};
```

Here is an example of defining a `vector` to hold strings:

```
vector<string> names = {"Joe", "Karen", "Lisa", "Jackie"};
```

The `vector` class overloads the `[]` operator, giving you the ability to access elements using a subscript, just as you would with an array. Program 17-4 gives a simple demonstration of creating and initializing a `vector` object, and displaying its elements.

**Program 17-4**

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main()
6  {
7      const int SIZE = 10;
8
9      // Define a vector to hold 10 int values.
10     vector<int> numbers(SIZE);
11
```

*(program continues)*

**Program 17-4** *(continued)*

```
12       // Store the values 0 through 9 in the vector.
13       for (int index = 0; index < numbers.size(); index++)
14          numbers[index] = index;
15
16       // Display the vector elements.
17       for (auto element : numbers)
18          cout << element << " ";
19       cout << endl;
20
21       return 0;
22    }
```

**Program Output**

```
0 1 2 3 4 5 6 7 8 9
```

Keep in mind the `[]` operator has its limitations. Specifically, you can use it only to access elements that already exist in a `vector`. In other words, you cannot add new elements to an existing `vector` using the `[]` operator. For example, the following code will cause an error at run time:

```
vector<int> numbers;              // Define an empty vector
numbers[0] = 99;                  // Error!
```

This code will cause an error because `numbers` is an empty `vector`. Therefore, the element `numbers[0]` does not exist. If you want to add new elements to a `vector`, you must use one of the member functions that exist for that purpose in the `vector` class. For example, the `push_back()` member function adds a new element to the end of the container. You simply call the function, passing the new element's value as an argument. The following code shows an example:

```
vector <int> numbers;
numbers.push_back(10);
numbers.push_back(20);
numbers.push_back(30);
```

The first statement creates an empty `vector` container named `numbers`. The statements that follow add the values 10, 20, and 30 to the `vector`.

You can use the `at()` member function to retrieve a `vector` element by its index with bounds checking. The following code shows an example:

```
vector<string> names = {"Joe", "Karen", "Lisa"};
cout << names.at(3) << endl;  // Throws an exception
```

The first statement in this code snippet defines a `vector` with three elements: "Joe" is at index 0, "Karen" is at index 1, and "Lisa" is at index 2. The second statement tries to display the element at index 3, which does not exist. As a result, the `at()` member function throws an `out_of_bounds` exception.

## Using an Iterator with a vector

The vector class supports the use of iterators of the following types:

- iterator
- const_iterator
- reverse_iterator
- const_reverse_iterator

Notice in Table 17-8 the vector class provides the following member functions for getting iterators: begin(), end(), cbegin(), cend(), rbegin(), rend(), crbegin(), and crend().

The following code shows an example of using an iterator to display all of the elements of a vector:

```
// Create a vector containing names.
vector<string> names = {"Joe", "Karen", "Lisa", "Jackie"};

// Create an iterator.
vector<string>::iterator it;

// Use the iterator to display each element in the vector.
for (it = names.begin(); it != names.end(); it++)
{
   cout << *it << endl;
}
```

We can simplify this code by auto declaring the iterator in the initialization expression of the for loop, as shown here:

```
// Create a vector containing names.
vector<string> names = {"Joe", "Karen", "Lisa", "Jackie"};

// Use an iterator to display each element in the vector.
for (auto it = names.begin(); it != names.end(); it++)
{
   cout << *it << endl;
}
```

## Inserting New Elements into a vector

You can use the insert member function to insert one or more new elements at a specified position in a vector. There are three overloaded versions of the insert member function shown in Table 17-8. Program 17-5 shows an example of how to use the first version.

**Program 17-5**

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main()
6  {
7      // Define a vector with 5 int values.
8      vector<int> numbers = {1, 2, 3, 4, 5};
9
```

*(program continues)*

**Program 17-5** *(continued)*

```
10      // Define an iterator pointing to the second element.
11      auto it = numbers.begin() + 1;
12
13      // Insert a new element with the value 99.
14      numbers.insert(it, 99);
15
16      // Display the vector elements.
17      for (auto element : numbers)
18         cout << element << " ";
19      cout << endl;
20
21      return 0;
22  }
```

**Program Output**

```
1 99 2 3 4 5
```

Let's take a closer look at the program:

- Line 8 defines a `vector` of `int`s, initialized with the values 1, 2, 3, 4, and 5.
- Line 11 defines an iterator that is pointing at the second element in the `vector`.
- Line 14 inserts a new element just before the one that `it` points to. The new element contains the value 99.
- The loop in lines 17 and 18 display all of the `vector`'s elements.

The second version of the `insert` member function allows you to insert multiple elements, each with the same value. The following code snippet shows an example:

```
// Define a vector with 5 int values.
vector<int> numbers = {1, 2, 3, 4, 5};

// Define an iterator pointing to the second element.
auto it = numbers.begin() + 1;

// Insert 3 new elements, each with the value 99.
numbers.insert(it, 3, 99);
```

The last statement inserts three elements, each with the value 99. After the statement executes, the `vector` will contain the following elements: `1 99 99 99 2 3 4 5`

The third version of the `insert` member function allows you to insert a range of elements, perhaps from a different container, into the `vector`. The function takes three iterators as arguments. The first iterator specifies the position where the new elements will be inserted. The second and third iterators mark the beginning and end of a range of elements to be inserted. Program 17-6 demonstrates the function by inserting a range of elements from one `vector` into another.

**Program 17-6**

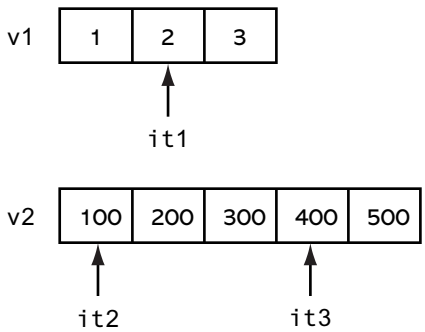
```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main()
6  {
7      // Define two vectors.
8      vector<int> v1 = {1, 2, 3};
9      vector<int> v2 = {100, 200, 300, 400, 500};
10
11     // Define iterators
12     auto it1 = v1.begin() + 1;  // Points at 2 in v1
13     auto it2 = v2.begin();      // Points at 100 in v2
14     auto it3 = v2.begin() + 3;  // Points at 400 in v2
15
16     // Insert a range of elements into v1.
17     v1.insert(it1, it2, it3);
18
19     // Display the elements of v1.
20     for (auto element : v1)
21        cout << element << " ";
22     cout << endl;
23
24     return 0;
25 }
```

**Program Output**

```
1 100 200 300 2 3
```

Program 17-6 defines two `vectors`: `v1` and `v2`. Line 12 defines an iterator, `it1`, pointing to the second element of `v1`. Line 13 defines an iterator, `it2`, pointing to the first element of `v2`. Line 14 defines an iterator, `it3`, pointing to the fourth element of `v2`. These iterator positions are illustrated in Figure 17-3. Then, line 17 inserts the range of elements from `it2` to `it3` (not including `it3`) into `v1`, before the element at `it1`.

**Figure 17-3** The iterators of Program 17-6



## Storing Objects of Your Own Classes as Values in a `vector`

In the examples previously shown, we stored strings and primitive values in `vectors`. It is often useful to store objects of classes that you have written in a `vector`. For example, the `Product` class, shown here, keeps information about a product. Specifically, it keeps the product's name and the number of units on hand.

### Contents of `Product.h`

```
 1  #ifndef PRODUCT_H
 2  #define PRODUCT_H
 3  #include <string>
 4  using namespace std;
 5
 6  class Product
 7  {
 8  private:
 9      string name;
10      int units;
11  public:
12      Product(string n, int u)
13      {  name = n;
14         units = u; }
15
16      void setName(string n)
17      {  name = n; }
18
19      void setUnits(int u)
20      {  units = u; }
21
22      string getName() const
23      {  return name; }
24
25      int getUnits() const
26      {  return units; }
27  };
28  #endif
```

Program 17-7 shows how we can define and initialize a `vector` to hold `Product` objects.

**Program 17-7**

```
 1  #include <iostream>
 2  #include <vector>
 3  #include "Product.h"
 4  using namespace std;
 5
 6  int main()
 7  {
 8      // Create a vector of Product objects.
 9      vector<Product> products =
10      {
11          Product("T-Shirt", 20),
12          Product("Calendar", 25),
13          Product("Coffee Mug", 30)
14      };
15
```

```
16      // Display the vector elements.
17      for (auto element : products)
18      {
19          cout << "Product: " << element.getName() << endl
20               << "Units: " << element.getUnits() << endl;
21      }
22
23      return 0;
24  }
```

**Program Output**

```
Product: T-Shirt
Units: 20
Product: Calendar
Units: 25
Product: Coffee Mug
Units: 30
```

Program 17-8 shows an example of using the `push_back` member function to add `Product` objects to an existing `vector`, then using an iterator to display the contents of the `vector`. Notice the use of the `->` operator in lines 25 and 26. In those lines, it points to a `Product` object, so the `->` operator is used to call the object's `getName()` and `getUnits()` member functions.

**Program 17-8**

```
 1  #include <iostream>
 2  #include <string>
 3  #include <vector>
 4  #include "Product.h"
 5  using namespace std;
 6
 7  int main()
 8  {
 9      // Create Product objects.
10      Product prod1("T-Shirt", 20);
11      Product prod2("Calendar", 25);
12      Product prod3("Coffee Mug", 30);
13
14      // Create a vector to hold the Products
15      vector<Product> products;
16
17      // Add the products to the vector.
18      products.push_back(prod1);
19      products.push_back(prod2);
20      products.push_back(prod3);
21
```

*(program continues)*

**Program 17-8** *(continued)*

```
22      // Use an iterator to display the vector contents.
23      for (auto it = products.begin(); it != products.end(); it++)
24      {
25          cout << "Product: " << it->getName() << endl
26               << "Units: " << it->getUnits() << endl;
27      }
28
29      return 0;
30  }
```

**Program Output**

```
Product: T-Shirt
Units: 20
Product: Calendar
Units: 25
Product: Coffee Mug
Units: 30
```

## Inserting Elements with the `emplace()` and `emplace_back()` Member Functions



Member functions such as `insert()` and `push_back()` can cause temporary objects to be created in memory while the insertion is taking place. For programs that do not make a lot of insertions, the overhead of creating temporary objects is not usually a problem. However, for programs that insert a lot of objects into containers, member functions such as `insert()` and `push_back()` can be inefficient.

To improve runtime performance, C++11 introduced a new family of member functions that use a technique known as *emplacement* to insert new elements. The process of emplacement avoids the creation of temporary objects in memory while a new object is being inserted into a container. As a result, the emplacement functions are much more efficient than functions such as `insert()` and `push_back()`, especially when a lot of objects are being inserted into a container.

The `vector` class provides two member functions that use emplacement: `emplace()` and `emplace_back()`.When you use one of these member functions, it is not necessary to instantiate, ahead of time, the object you are going to insert. Instead, you pass to the emplacement function any arguments that you would normally pass to the constructor of the object you are inserting. The emplacement function handles the construction of the object, forwarding the arguments to its constructor.

Program 17-9 shows an example of how we can use the `emplace_back()` member function to add `Product` objects to a `vector`. Notice we do not instantiate the `Product` class in the program. Instead, in lines 12 through 14 we pass the arguments that we would normally pass to the `Product` class constructor, to the `emplace_back()` member function. The `emplace_back()` member function constructs the objects in the `vector`. (This type of object construction is known as *in-place construction*.)

**Program 17-9**

```
1  #include <iostream>
2  #include <vector>
3  #include "Product.h"
4  using namespace std;
5
6  int main()
7  {
8      // Create a vector to hold Products.
9      vector<Product> products;
10
11     // Add Products to the vector.
12     products.emplace_back("T-Shirt", 20);
13     products.emplace_back("Calendar", 25);
14     products.emplace_back("Coffee Mug", 30);
15
16     // Use an iterator to display the vector contents.
17     for (auto it = products.begin(); it != products.end(); it++)
18     {
19         cout << "Product: " << it->getName() << endl
20              << "Units: " << it->getUnits() << endl;
21     }
22
23     return 0;
24 }
```

**Program Output**

```
Product: T-Shirt
Units: 20
Product: Calendar
Units: 25
Product: Coffee Mug
Units: 30
```

The `emplace()` member function constructs an object at a specified location in the `vector`. The first argument you pass to the `emplace()` member function is an iterator. The new element will be constructed at the position just before the element that the iterator points to. After the iterator, you pass the list of arguments that is to be forwarded to the new object's constructor. Program 17-10 shows an example.

**Program 17-10**

```
1  #include <iostream>
2  #include <vector>
3  #include "Product.h"
4  using namespace std;
5
```

*(program continues)*

**Program 17-10** *(continued)*

```
 6  int main()
 7  {
 8      // Create a vector to hold Products.
 9      vector<Product> products =
10      {
11          Product("T-Shirt", 20),
12          Product("Coffee Mug", 30)
13      };
14
15      // Get an iterator to the 2nd element.
16      auto it = products.begin() + 1;
17
18      // Insert another Product into the vector.
19      products.emplace(it, "Calendar", 25);
20
21      // Display the vector contents.
22      for (auto element : products)
23      {
24          cout << "Product: " << element.getName() << endl
25               << "Units: " << element.getUnits() << endl;
26      }
27
28      return 0;
29  }
```

**Program Output**

```
Product: T-Shirt
Units: 20
Product: Calendar
Units: 25
Product: Coffee Mug
Units: 30
```

Let's take a closer look at the program:

- Lines 9 through 13 create a `vector` named `products`, initialized with two `Product` objects.
- Line 16 defines an iterator, `it`, pointing to the second element in the `vector`.
- Line 19 inserts a new element into the `vector`, just before the element that `it` points to. The new element will be a `Product` object that is constructed, with the arguments "Calendar" and 25 passed as arguments to the constructor.

## The `capacity()`, `max_size()`, `shrink_to_fit()`, and `reserve()` Member Functions

The `vector` container uses a dynamically allocated array to hold its elements. When the underlying array is full, and another element is added to the `vector`, the `vector` has to increase the size of its underlying array. Typically, this means that a new array must be

allocated, all of the elements of the old array must be copied to the new array, and the old array must be deallocated.

To prevent this process from happening each time a new element is added, it is common for the `vector` class to allocate more memory than it needs. That way, a certain number of elements can be added without causing the reallocation process to take place. So, a `vector` object has two sizes: (1) the number of elements that are stored in the `vector`, and (2) the number of elements that can be stored in the vector without causing its underlying array to be reallocated.

You already know that the `size()` member function returns the number of elements that are currently stored in the `vector`. The `vector` class also has a `capacity()` member function that returns the number of elements that the container's underlying array can currently store, without allocating more memory. In addition, the `max_size()` member function returns the theoretical maximum number of elements that the `vector` can ever store.

The value returned from a `vector`'s `capacity()` member function will always be greaterthan or equal to the value returned by the `size()` member function. If you need to increase a `vector`'s capacity, you can call the `reserve()` member function to make the request. You pass an integer argument to specify the desired number of elements. If you need to decrease a `vector`'s capacity, you can call the `shrink_to_fit()` member function to make the request. The function takes no arguments, and it simply shrinks the `vector`'s underlying array so its capacity is the same as the `vector`'s size.



## Checkpoint

17.11 Write a statement that defines an empty `vector` object named `avect` that can hold strings.

17.12 Write a statement that defines a `vector` object named `avect` that can hold `int`s. The `vector` should have ten elements (initialized with the default value 0).

17.13 Write a statement that defines a `vector` object named `avect` that can hold `int`s. The `vector` should have 100 elements, each initialized with the value 1.

17.14 Write a statement that defines a `vector` object named `v1` that can hold `int`s. The `vector` should be a copy of another `vector` name `v2`.

17.15 What happens when you use an invalid index with the `vector` class's `at()` member function?

17.16 What is the difference between the `vector` class's `insert()` member function and `push_back()` member function?

17.17 If your program will be added a lot of objects to a `vector`, is it best to use the `inert()` member function, or the `emplace()` member function? Why?

17.18 Internally, how does a `vector` store its elements?

17.19 The `vector` class has a `size()` member function and a `capacity()` member function. What is the difference between these two member functions?

## 17.4 The map, multimap, and unordered_map Classes

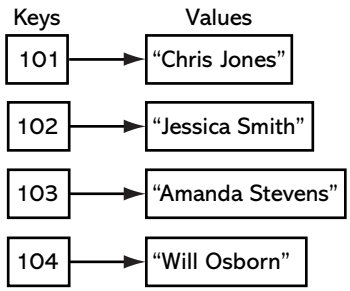
**CONCEPT: Each element in a map has two parts: a key and a value. Each key is associated with a specific value, and can be used to locate that value.**

A *map* is an associative container. Each element that is stored in a map has two parts: a *key* and a *value*. In fact, map elements are commonly referred to as *key–value pairs*. When you want to retrieve a specific value from a map, you use the key that is associated with that value. This is similar to the process of looking up a word in the dictionary, where the words are keys and the definitions are values.

For example, suppose each employee in a company has an ID number, and we want to write a program that lets us look up an employee's name by entering that employee's ID number. We could create a map in which each element contains an employee ID number as the key and that employee's name as the value. This is illustrated in Figure 17-4. If we know an employee's ID number, then we can retrieve that employee's name.

Another example would be a program that lets us enter a person's name and gives us that person's phone number. The program could use a map in which each element contains a person's name as the key and that person's phone number as the value. If we know a person's name, then we can retrieve that person's phone number.

**Figure 17-4** Key–value pairs






**NOTE:** Key–value pairs are often referred to as *mappings* because each key is mapped to a value.



VideoNote
**The map Container**

### The map Class

The STL provides a `map` class template you can use to implement a map container in your programs. The `map` class provides many member functions for storing elements, retrieving elements, deleting elements, iterating over the map, and much more.

To use the `map` class, you need to `#include` the `<map>` header file in your program. Then, you can define a `map` object using one of the `map` class constructors. Table 17-9 shows the general format of a `map` definition statement using each constructor. Table 17-10 lists many (but not all) of the `map` class's member functions.

**Table 17-9** map Definition Statements

| | |
|---|---|
| **Default Constructor** | `map<keyDatatype, valueDataType> name;`<br>Creates an empty `map` object. In the general format, *`keyDatatype`* is the data type of each element's key, *`valueDataType`* is the data type of each element's value, and *`name`* is the name of the `map`. |
| **Range Constructor** | `map<keyDatatype, valueDataType> name(iterator1, iterator2);`<br>Creates a `map` object that initially contains a range of values specified by two iterators. In the general format, *`keyDatatype`* is the data type of each element's key, *`valueDataType`* is the data type of each element's value, and *`name`* is the name of the `map`. The *`iterator1`* and *`iterator2`* arguments mark the beginning and end of a range of elements that will be stored in the map. |
| **Copy Constructor** | `map<keyDatatype, valueDataType> name(map2);`<br>Creates a `map` object that is a copy of another `map` or object. In the general format, *`keyDatatype`* is the data type of each element's key, *`valueDataType`* is the data type of each element's value, and *`name`* is the name of the `map`, and *`map2`* is the `map` to copy. |

**Table 17-10** Some of the map Member Functions

| Member Function | Description |
|---|---|
| `at(key)` | Returns a reference to the element containing the specified key. |
| `begin()` | Returns an iterator pointing to the first element in the container. |
| `cbegin()` | Returns a `const_iterator` to the first element in the container. |
| `cend()` | Returns a `const_iterator` pointing to the end of the container. |
| `clear()` | Erases all of the elements in the container. |
| `count(key)` | Returns the number of elements containing the specified key. |
| `crbegin()` | Returns a `const_reverse_iterator` pointing to the last element in the container. |
| `crend()` | Returns a `const_reverse_iterator` pointing to the first element in the container. |
| `emplace(key, value)` | Inserts a new element containing the specified *`key`* and *`value`* into the container. If an element with the specified key already exists, the function call does nothing. |
| `empty()` | Returns `true` if the container is empty, or `false` otherwise. |
| `end()` | Returns an iterator pointing to the end of the container (the position *`after`* the last element). |
| `erase(key)` | Erases the element containing the specified key. The function returns 1 if the element was erased, or 0 if no matching element was found. |
| `find(key)` | Searches for an element with the specified key. If the element is found, the function returns an iterator to it. If the element is not found, the function returns an iterator to the end of the `map`. |

*(table continues)*

**Table 17-10** *(continued)*

| Member Function | Description |
|---|---|
| `insert(pair)` | Inserts a `pair` object as an element to the map. If an element with the specified key already exists, the function call does nothing. |
| `lower_bound(key)` | Returns an iterator pointing to the first element with a key that is equal to or greater than *key*. |
| `max_size()` | Returns the theoretical maximum size of the container. |
| `rbegin()` | Returns a `reverse_iterator` pointing to the last element in the container. |
| `rend()` | Returns a `reverse_iterator` pointing to the first element in the container. |
| `size()` | Returns the number of elements in the container. |
| `swap(second)` | The *second* argument must be a `map` object of the same type as the calling object. The function swaps the contents of the calling object and the *second* object. |
| `upper_bound(key)` | Returns an iterator pointing to the first element with a key that is greater than *key*. |

Here is an example of how you might define a `map` container to hold employee ID numbers (as `int`s) and their corresponding employee names (as `string`s):

```
map<int, string> employees;
```

This statement defines a `map` container named `employees`. Inside the angled brackets <>, the first data type (`int`) is the type of each key, which in this case are the employee ID numbers. The second data type (`string`) is the type of the corresponding values, which in this case are the employee names. So, each element of this container will be a key–value pair where the key is an `int` and the value is a `string`.

The keys in a `map` container must be unique. No two elements in a `map` container can have the same key value.

## Initializing a Map

You can initialize a `map` with an initialization list. The following code shows an example:

```
map<int, string> employees =
  {{101, "Chris Jones"}, {102, "Jessica Smith"},
   {103, "Amanda Stevens"}, (104, "Will Osborn"}};
```

This code creates a `map` container named `employees`, and initializes it as follows:

- The first element is `{101, "Chris Jones"}`. In this element, 101 is the key and "Chris Jones" is the value.
- The second element is `{102, "Jessica Smith"}`. In this element, 102 is the key and "Jessica Smith" is the value.

- The third element is {103, "Amanda Stevens"}. In this element, 103 is the key and "Amanda Stevens" is the value.
- The fourth element is {104, "Will Osborn"}. In this element, 104 is the key and "Will Osborn" is the value.

Notice inside the initialization list:

- The initialization list is enclosed in a set of curly braces {}.
- Each element is enclosed in its own set of curly braces, with the key and the value separated by a comma.
- The elements are separated by commas.

As previously mentioned, the keys in a map must be unique. If two or more elements in an initialization list have the same key, only the first will be added to the map. The others will be ignored.

## Adding Elements to an Existing Map

The map class overloads the [] operator, giving you the ability to add new elements to a map with an assignment statement in the following general format:

```
mapName[key] = value;
```

In the general format, *mapName* is the name of the map, and *key* is a key. If *key* already exists in the map, its associated value will be changed to *value*. If the *key* does not exist, it will be added to the map, along with *value* as its associated value. The following code shows an example:

```
map<int, string> employees;
employees[110] = "Beth Young";
employees[111] = "Jake Brown";
employees[112] = "Emily Davis";
```

The first statement creates an empty map container named employees. The statements that follow then add the following elements to the map:

- Key = 110, Value = "Beth Young"
- Key = 111, Value = "Jake Brown"
- Key = 112, Value = "Emily Davis"

Remember, the keys in a map must be unique. If you assign a new value to an existing key, the new value will replace the previously stored value. For example, look at the following code:

```
map<int, string> employees;
employees[110] = "Beth Young";
employees[110] = "Jake Brown";
```

The first statement creates an empty map container named employees. The second statement adds an element with the key 110 and the value "Beth Young". The third statement assigns the value "Jake Brown" to the element with the key 110.

## Adding Elements with the `insert()` Member Function

When an element is stored in a `map`, it is stored as object of the `pair` type. The `pair` type is a `struct` that has two member variables: `first` and `second`. The element's key is stored in the `first` member variable, and the element's value is stored in the `second` member variable.

The `map` class provides an `insert()` member function that adds a `pair` object as an element to the map. You can use the STL function `make_pair` to construct a `pair` object, as shown in the following code:

```
map<int, string> employees;
employees.insert(make_pair(110, "Beth Young"));
employees.insert(make_pair(111, "Jake Brown"));
employees.insert(make_pair(112, "Emily Davis"));
```

The first statement creates an empty `map` container named `employees`. The statements that follow then add the following elements to the `map`:

- Key = 110, Value = "Beth Young"
- Key = 111, Value = "Jake Brown"
- Key = 112, Value = "Emily Davis"

If the element that you are inserting with the `insert()` member function has the same key as an existing element, the function will not insert the new element.



**NOTE:** The `pair struct` and the `make_pair` function template are declared in the `<utility>` header file. If you have included the `<map>` header file, the `<utility>` header file will be automatically included as well.

## Adding Elements with the `emplace()` Member Function

The `map` class provides an `emplace()` member function that constructs a new element in the container. You simply call the member function, passing the new element's key and value as arguments. The following code shows an example:

```
map<int, string> employees;
employees.emplace(110, "Beth Young");
employees.emplace(111, "Jake Brown");
employees.emplace(112, "Emily Davis");
```

The first statement creates an empty `map` container named `employees`. The statements that follow then add the following elements to the `map`:

- Key = 110, Value = "Beth Young"
- Key = 111, Value = "Jake Brown"
- Key = 112, Value = "Emily Davis"

If the element that you are inserting with the `emplace()` member function has the same key as an existing element, the function will not insert the new element.

## Retrieving Values from a Map

To retrieve a value from a map, you call the at() member function, passing the key that is associated with the desired value. If the specified key exists in the container, the function returns its associated value. If the key does not exist, the function throws an exception. The following code shows an example:

```
// Create a map containing employee IDs and names.
map<int, string> employees =
  {{101, "Chris Jones"}, {102, "Jessica Smith"},
   {103, "Amanda Stevens"}, (104, "Will Osborn"}};

// Retrieve a value from the map.
cout << employees.at(103) << endl;
```

This code retrieves and displays the value that is associated with the key 103 from the map. The code will display the string "Amanda Stevens".

If the key you pass to the at() member function does not exist in the map, the at() member function will throw an exception. To prevent this, you should first call the count() member function to determine whether the key exists in the map. You call the count() member function, passing a key as the argument. If the specified key exists in the container, the function returns 1. If the key does not exist, the function returns 0. The following code shows an example:

```
// Create a map containing employee IDs and names.
map<int, string> employees =
  {{101, "Chris Jones"}, {102, "Jessica Smith"},
   {103, "Amanda Stevens"}, (104, "Will Osborn"}};

// Retrieve a value from the map.
if (employees.count(103))
   cout << employees.at(103) << endl;
else
   cout << "Employee not found.\n";
```

## Deleting Elements

You can delete, or erase, an element from a map using the erase() member function. You call the erase() member function, passing the key that is associated with the element you want to erase. The following code shows an example:

```
// Create a map containing employee IDs and names.
map<int, string> employees =
  {{101, "Chris Jones"}, {102, "Jessica Smith"},
   {103, "Amanda Stevens"}, (104, "Will Osborn"}};

// Delete the employee with the ID 102.
employees.erase(102);
```

This code creates a map container with four elements. Then, it deletes the element with the key 102 (and the value "Jessica Smith"). The map then contains only three elements.

The erase() member function returns 1 if the specified element was successfully erased, or 0 if it was not found.

## Iterating Over a Map with the Range-Based for Loop

The range-based for loop is a convenient way to iterate over all of the elements in a map. The following code shows an example of how to display all of the elements in a map:

```
// Create a map containing employee IDs and names.
map<int, string> employees =
  {{101, "Chris Jones"}, {102, "Jessica Smith"},
   {103, "Amanda Stevens"}, (104, "Will Osborn"}};

// Display each element.
for (auto element : employees)
{
   cout << "ID:" << element.first << "\tName:" << element.second << endl;
}
```

Notice we used the auto key word in the declaration of the range variable, element. The auto key word tells the compiler to determine the variable's data type from the context in which it is being used. If we had written the actual data type of the element variable, the loop would have looked like this:

```
for (pair<int, string> element : employees)
{
   cout << "ID: " <<element.first << "\tName:" <<element.second << endl;
}
```

As previously mentioned, elements are stored in a map container as objects of the pair type. The pair type is a struct that has two member variables: first and second. The element's key is stored in the first member variable, and the element's value is stored in the second member variable.

## Using an Iterator with a Map

You can use a bidirectional iterator to access the elements of a map. The map class has a begin() member function that returns an iterator pointing to the first element in the map, and an end() member function that returns an iterator pointing to the end of the map (the position *after* the last element). Program 17-11 shows an example of using an iterator to display all of the elements of a map.

**Program 17-11**

```
1  // This program demonstrates an iterator with a map.
2  #include <iostream>
3  #include <string>
4  #include <map>
5  using namespace std;
6
7  int main()
8  {
9      // Create a map containing employee IDs and names.
10     map<int, string> employees =
11        { {101,"Chris Jones"}, {102,"Jessica Smith"},
12          {103,"Amanda Stevens"},{104,"Will Osborn"} };
13
```

```
14      // Create an iterator.
15      map<int, string>::iterator iter;
16
17      // Use the iterator to display each element in the map.
18      for (iter = employees.begin(); iter != employees.end(); iter++)
19      {
20          cout << "ID: " << iter->first
21               << "\tName: " << iter->second << endl;
22      }
23
24      return 0;
25  }
```

**Program Output**

```
ID: 101 Name: Chris Jones
ID: 102 Name: Jessica Smith
ID: 103 Name: Amanda Stevens
ID: 104 Name: Will Osborn
```

Let's take a closer look at Program 17-11:

- Lines 10 through 12 define a `map` container named `employees`, and initialize it with four elements. Notice the map's type is `map<int, string>`.
- Line 15 defines an iterator that will work with an object of the type `map<int, string>`. The iterator's name is `iter`.
- The `for` loop that appears in lines 18 through 22 uses the iterator to step through the map container.
  - The loop's initialization expression `iter = employees.begin()` causes the iterator to point to the first element in the map.
  - The test expression `iter != employees.end()` causes the loop to execute as long as the iterator does not point to the end of the map.
  - The update expression increments the iterator, moving it to the next element in the map.
  - The body of the loop, in lines 20 and 21, displays the contents of the element to which the iterator is currently pointing. Remember, each element of the map is a `pair` object, with the key stored in the `first` member, and the value stored in the `second` member.



**NOTE:** When you access the elements of a `map` from first to last, you access them in order of their keys.



**NOTE:** The `end()` member function returns an iterator pointing to the end of the `map`, but it does not point to an actual element. It points to the position where an additional element would exist, if it appeared after the last element.

The map class also has a find() member function that searches for an element with a specified key. If the element is found, the find() function returns an iterator to it. If the element is not found, the find() function returns an iterator to the end of the map. The following code shows an example:

```
// Create a map containing employee IDs and names.
map<int, string> employees =
  {{101, "Chris Jones"}, {102, "Jessica Smith"},
   {103, "Amanda Stevens"}, (104, "Will Osborn"}};

// Create an iterator.
map<int, string>::iterator iter;

// Find employee 103.
iter = employees.find(103);

// Display the employee data.
if (iter != employees.end())
{
   cout << "ID: " << iter->first
        << "\tName: " << iter->second << endl;
}
else
{
   cout << "Employee not found.\n";
}
```

## Storing vectors as Values in a map

In Program 17-12, we define a map in which the keys are student names (as strings), and the values are vectors containing test scores.

**Program 17-12**

```
 1  #include <iostream>
 2  #include <string>
 3  #include <vector>
 4  #include <map>
 5  using namespace std;
 6
 7  int main()
 8  {
 9      // Create vectors to hold test scores.
10      vector<int> student1Scores = {88, 92, 100};
11      vector<int> student2Scores = {95, 74, 81};
12      vector<int> student3Scores = {72, 88, 91};
13      vector<int> student4Scores = {70, 75, 78};
14
15      // Create a map to hold all the test scores.
16      map<string, vector<int>> testScores;
17      testScores["Kayla"] = student1Scores;
18      testScores["Luis"] = student2Scores;
19      testScores["Sophie"] = student3Scores;
20      testScores["Ethan"] = student4Scores;
21
```

```
22      // Display each student's test scores.
23      for (auto element : testScores)
24      {
25         // Display the student name.
26         cout << "Student: " << element.first << endl;
27
28         // Display the student's test scores.
29         for (int i = 0; i < element.second.size(); i++)
30         {
31            cout << "\t" << element.second[i] << endl;
32         }
33      }
34      return 0;
35  }
```

**Program Output**

```
Student: Ethan
         70
         75
         78
Student: Kayla
         88
         92
         100
Student: Luis
         95
         74
         81
Student: Sophie
         72
         88
         91
```

Let's take a closer look at Program 17-12:

- Lines 10 through 13 create four `vectors`, each holding a different student's test scores.
- Line 16 defines a `map` container named `testScores`. Notice the `map`'s type is `map<string, vector<int>>`. This indicates the keys will be of the `string` type, and the values will be of the `vector<int>` type.
- Lines 17 through 20 add the following elements to the map:

  Key = "Kayla", Value = `student1Scores`, which is a `vector<int>` object
  Key = "Luis", Value = `student2Scores`, which is a `vector<int>` object
  Key = "Sophie", Value = `student3Scores`, which is a `vector<int>` object
  Key = "Ethan", Value = `student4Scores`, which is a `vector<int>` object

- The range-based `for` loop that begins in line 23 iterates over each element in the map. Remember that the range variable, `element`, is an object of the `pair` data type. The `element` object's `first` member variable is the element's key, and its `second` member variable is the element's value. So, the expression `element.first` is a string, and the expression `element.second` is a `vector<int>` object. In line 31, the expression `element.second[i]` is an element in the `vector`.

Program 17-12 illustrates the process of storing `vectors` as values in a `map`, but the program can be simplified in two ways: (1) When we define the `map`, we can use an initialization list that also creates the `vector` objects. That will allow us to eliminate the `vector` definition statements in lines 10 through 13. (2) We can rewrite the inner `for` loop in lines 29 through 32 as a range-based `for` loop. Program 17-13 shows the program with these changes.

**Program 17-13**

```
 1  #include <iostream>
 2  #include <string>
 3  #include <vector>
 4  #include <map>
 5  using namespace std;
 6
 7  int main()
 8  {
 9       // Create vectors to hold test scores.
10       map<string, vector<int>> testScores =
11          { {"Kayla",  vector<int> {88, 92, 100 }},
12            {"Luis",   vector<int> {95, 74, 81 }},
13            {"Sophie", vector<int> {72, 88, 91 }},
14            {"Ethan",  vector<int> {70, 75, 78 }} };
15
16       // Display each student's test scores.
17       for (auto element : testScores)
18       {
19          // Display the student name.
20          cout << "Student: " << element.first << endl;
21
22          // Display the student's test scores.
23          for (auto score : element.second)
24          {
25             cout << "\t" << score << endl;
26          }
27       }
28       return 0;
29  }
```

**Program Output**

```
Student: Ethan
         70
         75
         78
Student: Kayla
         88
         92
         100
Student: Luis
         95
         74
         81
Student: Sophie
         72
         88
         91
```

## Storing Objects of Your Own Classes as Values in a map

In the examples previously shown, we stored strings, primitive values, and vectors in maps. It is often useful to store objects of classes that you have written in a map. For example, suppose you are writing a program to keep a contact list, and you want to search for a person's contact information by entering that person's name. You could create a map that contains an element for each person in the contact list. In each element, the key is a person's name, and the value is an object containing that person's contact data. You can search the map, using a person's name as the key, and retrieve the object containing that person's contact data.

If you want to store an object as a value in a map, there is one requirement for that object's class: It must have a default constructor. For example, the Contact class shown here holds a person's name and e-mail address. Because it has a default constructor (in lines 12 through 14), objects of this class can be stored as values in a map.

### Contents of Contact.h

```
1  #ifndef CONTACT_H
2  #define CONTACT_H
3  #include <string>
4  using namespace std;
5
6  class Contact
7  {
8  private:
9      string name;
10     string email;
11 public:
12     Contact()
13     {  name = "";
14        email = ""; }
15
16     Contact(string n, string em)
17     {  name = n;
18        email = em; }
19
20     void setName(string n)
21     {  name = n; }
22
23     void setEmail(string em)
24     {  email = em; }
25
26     string getName() const
27     {  return name; }
28
29     string getEmail() const
30     {  return email; }
31 };
32 #endif
```

Program 17-14 demonstrates how to store objects of the Contact class in a map. The program creates three Contact objects, then stores them in a map, using each Contact object's name field as the key. The user is prompted to enter a name to search for.

**Program 17-14**

```
#include <iostream>
#include <string>
#include <map>
#include "Contact.h"
using namespace std;

int main()
{
     string searchName;    // The name to search for

     // Create some Contact objects
     Contact contact1("Ashley Miller", "amiller@faber.edu");
     Contact contact2("Jacob Brown", "jbrown@gotham.edu");
     Contact contact3("Emily Ramirez", "eramirez@coolidge.edu");

     // Create a map to hold the Contact objects.
     map<string, Contact> contacts;

     // Create an iterator for the map.
     map<string, Contact>::iterator iter;

     // Add the contact objects to the map.
     contacts[contact1.getName()] = contact1;
     contacts[contact2.getName()] = contact2;
     contacts[contact3.getName()] = contact3;

     // Get the name to search for.
     cout << "Enter a name: ";
     getline(cin, searchName);

     // Search for the name.
     iter = contacts.find(searchName);

     // Display the results.
     if (iter != contacts.end())
     {
          cout << "Name: " << iter->second.getName() << endl;
          cout << "Email: " << iter->second.getEmail() << endl;
     }
     else
     {
          cout << "Contact not found.\n";
     }

     return 0;
}
```

**Program Output (with Example Input Shown in Bold)**

```
Enter a name: Emily Ramirez Enter
Name: Emily Ramirez
Email: eramirez@coolidge.edu
```

**Program Output (with Example Input Shown in Bold)**

```
Enter a name: Billy Clark Enter
Contact not found.
```

Let's take a closer look at Program 17-14:

- Lines 12 through 14 create three `Contact` objects named `contact1`, `contact2`, and `contact3`.
- Line 17 defines a `map` named `contacts`. Inside the angled brackets `<>`, the first data type (`string`) is the type of each key. The second data type (`Contact`) is the type of the corresponding values. So, each element of this container will be a key–value pair where the key is a `string` and the value is a `Contact` object.
- Line 20 defines an iterator that can be used with the `contacts` map.
- Line 23 adds an element to the `contacts` map. The key is the string "Ashley Miller", and the value is the `contact1` object.
- Line 24 adds another element to the `contacts` map. The key is the string "Jacob Brown", and the value is the `contact2` object.
- Line 25 adds another element to the `contacts` map. The key is the string "Emily Ramirez", and the value is the `contact3` object.
- Lines 28 and 29 prompt the user to enter a name to search for. The user's input is stored in `searchName`.
- Line 32 calls the `contacts.find()` member function to search for the name that was entered by the user. If the name is found as a key in the map, the member function returns an iterator to the element. If the name is not found, the member function returns an iterator to the end of the map.
- If the name was found, the statements in lines 37 and 38 display the name and e-mail address for the contact. Remember, the iterator is pointing to a `pair` object. The `pair` object's `first` member variable is the element's key, and the `pair` object's `second` member variable is the element's value. So, in lines 37 and 38, the expression `iter->second` references a `Contact` object. The expression `iter->second.getName()` returns that object's `name` field, and the expression `iter->second.getEmail()` returns that object's `email` field.

Program 17-15 shows another example in which we use a range-based `for` loop to iterate over a `map` containing `Contact` objects as values.

**Program 17-15**

```
1  #include <iostream>
2  #include <string>
3  #include <map>
4  #include "Contact.h"
5  using namespace std;
6
```

*(program continues)*

**Program 17-15** *(continued)*

```
 7  int main()
 8  {
 9      // Create some Contact objects
10      Contact contact1("Ashley Miller", "amiller@faber.edu");
11      Contact contact2("Jacob Brown", "jbrown@gotham.edu");
12      Contact contact3("Emily Ramirez", "eramirez@coolidge.edu");
13
14      // Create a map to hold the Contact objects.
15      map<string, Contact> contacts;
16
17      // Add the contact objects to the map.
18      contacts[contact1.getName()] = contact1;
19      contacts[contact2.getName()] = contact2;
20      contacts[contact3.getName()] = contact3;
21
22      // Display all objects in the map.
23      for (auto element : contacts)
24      {
25         cout << element.second.getName() << "\t"
26              << element.second.getEmail() << endl;
27      }
28
29      return 0;
30  }
```

**Program Output**

```
Ashley Miller    amiller@faber.edu
Emily Ramirez    eramirez@coolidge.edu
Jacob Brown      jbrown@gotham.edu
```

In lines 25 and 26, the range variable `element` is a `pair` object. The `pair` object's `first` member variable is the element's key, and the `pair` object's `second` member variable is the element's value. So, the expression `element.second` references a `Contact` object. The expression `element.second.getName()` returns that object's `name` field, and the expression `element.second.getEmail()` returns that object's `email` field.

Program 17-15 illustrates the process of using your own objects as values in a `map`, but the program can be simplified. When we define the `map`, we can use an initialization list that also creates the `Contact` objects. That will allow us to eliminate the `Contact` definition statements in lines 10 through 12. Program 17-16 shows the program with these changes.

**Program 17-16**

```
1  #include <iostream>
2  #include <string>
3  #include <map>
4  #include "Contact.h"
5  using namespace std;
6
```

```
 7  int main()
 8  {
 9      // Create a map holding the Contact objects.
10      map<string, Contact> contacts =
11       {{ "Ashley Miller", Contact("Ashley Miller", "amiller@faber.edu") },
12        { "Jacob Brown",   Contact("Jacob Brown", "jbrown@gotham.edu") },
13        { "Emily Ramirez", Contact("Emily Ramirez", "eramirez@coolidge.edu")}
14       };
15
16      // Display all objects in the map.
17      for (auto element : contacts)
18      {
19         cout << element.second.getName() << "\t"
20              << element.second.getEmail() << endl;
21      }
22
23      return 0;
24  }
```

**Program Output**

```
Ashley Miller    amiller@faber.edu
Emily Ramirez    eramirez@coolidge.edu
Jacob Brown      jbrown@gotham.edu
```

## Using an Object of Your Own Class as a Key

You can use objects of a class that you have written as keys in a `map`, as long as the class has overloaded the `<` operator. For example, look at the `Customer` class shown here:

**Contents of `Customer.h`**

```
 1  #ifndef CUSTOMER_H
 2  #define CUSTOMER_H
 3  #include<string>
 4  using namespace std;
 5
 6  class Customer
 7  {
 8  private:
 9      int custNumber;
10      string name;
11  public:
12      Customer(int cn, string n)
13      {  custNumber = cn;
14         name = n; }
15
16      void setCustNumber(int cn)
17      {  custNumber = cn; }
18
19      void setName(string n)
20      {  name = n; }
21
22      int getCustNumber() const
23      {  return custNumber; }
24
```

```
25     string getName() const
26     {  return name; }
27
28     bool operator < (const Customer &right) const
29     {  bool status = false;
30
31        if (custNumber < right.custNumber)
32           status = true;
33
34        return status; }
35  };
36  #endif
```

The `Customer` class holds two pieces of data about a customer: the customer number (`custNumber`) and the customer's name (`name`). The overloaded `<` operator (in lines 28 through 35) returns `true` if the `custNumber` member is less than the parameter's `custNumber` member. Otherwise, it returns `false`.

Suppose we are writing a program that assigns seats in a theater to customers, and we want to use `Customer` objects as the keys in a `map`. Program 17-17 shows an example. In line 15, the program defines a `map` that uses `Customer` objects as keys, and strings as values.

**Program 17-17**

```
1  #include <iostream>
2  #include <string>
3  #include <map>
4  #include "Customer.h"
5  using namespace std;
6
7  int main()
8  {
9     // Create some Customer objects.
10    Customer customer1(1001, "Sarah Scott");
11    Customer customer2(1002, "Austin Hill");
12    Customer customer3(1003, "Megan Cruz");
13
14    // Create a map to hold the seat assignments.
15    map<Customer, string> assignments;
16
17    // Use the map to store the seat assignments.
18    assignments[customer1] = "1A";
19    assignments[customer2] = "2B";
20    assignments[customer3] = "3C";
21
22    // Display all objects in the map.
23    for (auto element : assignments)
24    {
25       cout << element.first.getName() << "\t"
26            << element.second << endl;
27    }
28
29    return 0;
30 }
```

**Program Output**

```
Sarah Scott      1A
Austin Hill      2B
Megan Cruz       3C
```

Program 17-17 illustrates the process of using your own objects as keys in a map, but the program can be simplified. When we define the map, we can use an initialization list that also creates the Customer objects. That will allow us to eliminate the Customer definition statements in lines 10 through 12. Program 17-18 shows the program with these changes.

**Program 17-18**

```
 1  #include <iostream>
 2  #include <string>
 3  #include <map>
 4  #include "Customer.h"
 5  using namespace std;
 6
 7  int main()
 8  {
 9      // Create a map to hold the seat assignments.
10      map<Customer, string> assignments =
11        { { Customer(1001, "Sarah Scott"), "1A"},
12          { Customer(1002, "Austin Hill"), "2B"},
13          { Customer(1003, "Megan Cruz"), "3C" } };
14
15      // Display all objects in the map.
16      for (auto element : assignments)
17      {
18         cout << element.first.getName() << "\t"
19              << element.second << endl;
20      }
21
22      return 0;
23  }
```

**Program Output**

```
Sarah Scott      1A
Austin Hill      2B
Megan Cruz       3C
```

## The unordered_map Class



Beginning in C++11, the STL provides a class template named unordered_map that is similar to the map class, except in two regards: (1) the keys in an unordered_map are not sorted in any particular way, and (2) the unordered_map class has better performance. If you will be making a lot of searches on a large number of elements, and you are not concerned with retrieving them in key order, you should probably use the unordered_map class instead of the map class.

To use the unordered_map class, you must write the #include <unordered_map> directive in your program. The unordered_map class has a similar set of member functions as the map class (many of which are listed in Table 17-10), and in most cases, working with an unordered_map is just like working with a map. The following code shows an unordered_map being initialized with elements, then an element being retrieved and displayed:

```
// Create an unordered_map containing employee IDs and names.
unordered_map<int, string> employees =
  {{101, "Chris Jones"}, {102, "Jessica Smith"},
   {103, "Amanda Stevens"}, (104, "Will Osborn"}};

// Retrieve a value from the map.
cout << employees.at(103) << endl;
```

The following code shows an example of using a range-based for loop to display all of the elements in a map:

```
// Create an unordered_map containing employee IDs and names.
unordered_map <int, string> employees =
  {{101, "Chris Jones"}, {102, "Jessica Smith"},
   {103, "Amanda Stevens"}, (104, "Will Osborn"}};

// Display each element.
for (auto element : employees)
{
   cout << "ID:" << element.first << "\tName:" << element.second << endl;
}
```



**NOTE:** The differences between the unordered_map class and the map class are a result of the way each internally stores its elements. The map class uses a data structure known as a *binary tree*, and the unordered_map class uses a *hash table*.

## The multimap Class

The multimap class lets you create a map container in which multiple elements can have the same key. In other words, duplicate keys are allowed in a multimap. To use the multimap class, you will need to write the #include<map> directive in your program. Table 17-11 lists many (but not all) of the multimap class's member functions.

**Table 17-11** Some of the multimap Member Functions

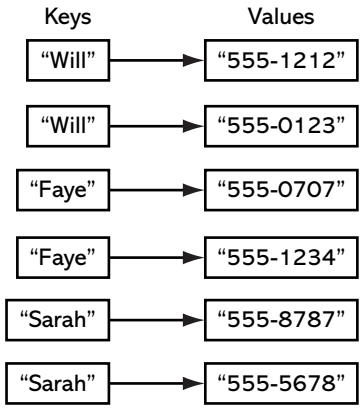
| Member Function | Description |
|---|---|
| begin() | Returns an iterator to the first element in the container. |
| cbegin() | Returns a const_iterator to the first element in the container. |
| cend() | Returns a const_iterator pointing to the end of the container. |
| clear() | Erases all of the elements in the container. |
| count(*key*) | Returns the number of elements containing the specified key. |

**Table 17-11** *(continued)*

| Member Function | Description |
|---|---|
| `crbegin()` | Returns a `const_reverse_iterator` pointing to the last element in the container. |
| `crend()` | Returns a `const_reverse_iterator` pointing to the first element in the container. |
| `emplace(key, value)` | Inserts a new element containing the specified *key* and *value* into the container. |
| `empty()` | Returns `true` if the container is empty, or `false` otherwise. |
| `end()` | Returns an iterator to the last element in the container. |
| `equal_range(key)` | Returns a `pair` object. The `pair` object's `first` member is an iterator pointing to the first element in the `multimap` that matches the specified key. The `pair` object's `second` member is an iterator pointing to the position *after* the last element that matches the specified key. If the specified key is not found, both iterators will point to the element that would naturally appear after the element that was searched for. |
| `erase(key)` | Erases all of the elements containing the specified key. The function returns the number of elements that were erased. |
| `find(key)` | Searches for an element with the specified key, and returns an iterator to the first matching element. If the element is not found, the function returns an iterator to the end of the container. |
| `insert(pair)` | Inserts a `pair` object as an element to the map. |
| `lower_bound(key)` | Returns an iterator pointing to the first element with a key that is equal to or greater than *key*. |
| `max_size()` | Returns the theoretical maximum size of the container. |
| `rbegin()` | Returns a `reverse_iterator` pointing to the last element in the container. |
| `rend()` | Returns a `reverse_iterator` pointing to the first element in the container. |
| `size()` | Returns the number of elements in the container. |
| `swap(second)` | The *second* argument must be a `map` object of the same type as the calling object. The function swaps the contents of the calling object and the *second* object. |
| `upper_bound(key)` | Returns an iterator pointing to the first element with a key that is greater than *key*. |

For example, in a phone book application, you could store names and phone numbers in a `multimap` container, using the names as keys and the phone numbers as values. If a person has multiple phone numbers, you could store multiple elements using that person's name as the key. Figure 17-5 illustrates this concept.

**Figure 17-5** Duplicate keys stored in a `multimap` container




In Figure 17-5, there are two elements with the key "Will", two elements with the key "Faye", and two elements with the key "Sarah". Program 17-19 shows how to define a `multimap` and initialize it with these elements.

**Program 17-19**

```
#include <iostream>
#include <string>
#include <map>
using namespace std;

int main()
{
   // Define a phonebook multimap.
   multimap<string, string> phonebook =
      { {"Will", "555-1212"},  {"Will", "555-0123"},
        {"Faye", "555-0707"},  {"Faye", "555-1234"},
        {"Sarah", "555-8787"}, {"Sarah", "555-5678"} };

   // Display the elements in the multimap.
   for (auto element : phonebook)
   {
      cout << element.first << "\t"
           << element.second << endl;
   }
   return 0;
}
```

**Program Output**

```
Faye    555-0707
Faye    555-1234
Sarah   555-8787
Sarah   555-5678
Will    555-1212
Will    555-0123
```

As with the map class, the elements of a multimap are ordered. When you iterate over a multimap, you retrieve its elements in order of their keys. The multimap class also provides many of the same member functions as the map class. There are some differences between the multimap class and the map class, however, and we will focus on many of those differences in the rest of this section.

## Adding Elements to a multimap

Unlike the map class, the multimap class does not overload the [] operator. So, you cannot write an assignment statement to add a new element to a multimap. Instead, you can use either the emplace() member function, or the insert() member function.

The multimap class's emplace() member function adds a new element to the container. You simply call the member function, passing the new element's key and value as arguments. The following code shows an example:

```
multimap<string, string> phonebook;
phonebook.emplace("Will", "555-1212");
phonebook.emplace("Will", "555-0123");
phonebook.emplace("Faye", "555-0707");
phonebook.emplace("Faye", "555-1234");
phonebook.emplace("Sarah", "555-8787");
phonebook.emplace("Sarah", "555-5678");
```

The first statement creates an empty multimap container named phonebook. The statements that follow use the emplace() member function to add the elements shown in Figure 17-5.

If the element that you are inserting with the emplace() member function has the same key as an existing element, the function adds a new element with the same key.

The insert() member function adds a pair object as an element to the multimap. You can use the STL function make_pair to construct a pair object, as shown in the following code:

```
multimap<string, string> phonebook;
phonebook.insert(make_pair("Will", "555-1212"));
phonebook.insert(make_pair("Will", "555-0123"));
phonebook.insert(make_pair("Faye", "555-0707"));
phonebook.insert(make_pair("Faye", "555-1234"));
phonebook.insert(make_pair("Sarah", "555-8787"));
phonebook.insert(make_pair("Sarah", "555-5678"));
```

The first statement creates an empty multimap container named phonebook. The statements that follow use the insert() member function to add the elements shown in Figure 17-5.

If the element that you are inserting with the insert() member function has the same key as an existing element, the function adds a new element with the same key.

## Getting the Number of Elements with a Specified Key

The multimap class's count() member function accepts a key as its argument, and returns the number of elements that match the specified key. Program 17-20 demonstrates.

**Program 17-20**

```
1  #include <iostream>
2  #include <string>
3  #include <map>
4  using namespace std;
5
6  int main()
7  {
8     // Define a phonebook multimap.
9     multimap<string, string> phonebook =
10       { {"Will", "555-1212"},  {"Will", "555-0123"},
11         {"Faye", "555-0707"},  {"Faye", "555-1234"},
12         {"Sarah", "555-8787"}, {"Sarah", "555-5678"} };
13
14    // Display the number of elements that match "Faye".
15    cout << "Faye has " << phonebook.count("Faye") << " elements.\n";
16    return 0;
17 }
```

**Program Output**

```
Faye has 2 elements.
```

## Retrieving the Elements with a Specified Key

Like the `map` class, the `multimap` class has a `find()` member function that searches for an element with a specified key. If the key is found, the `find()` function returns an iterator to the first element matching it. If the element is not found, the `find()` function returns an iterator to the end of the `map`.

If you want to retrieve all of the elements matching a specified key, use the `multimap` class's `equal_range` member function. The `equal_range` member function returns a `pair` object. The `pair` object's `first` member is an iterator pointing to the first element in the `multimap` that matches the specified key. The `pair` object's `second` member is an iterator pointing to the position *after* the last element that matches the specified key. Program 17-21 demonstrates how to use the function.

**Program 17-21**

```
1  #include <iostream>
2  #include <string>
3  #include <map>
4  using namespace std;
5
6  int main()
7  {
8     // Define a phonebook multimap.
9     multimap<string, string> phonebook =
10       { {"Will", "555-1212"},  {"Will", "555-0123"},
11         {"Faye", "555-0707"},  {"Faye", "555-1234"},
12         {"Sarah", "555-8787"}, {"Sarah", "555-5678"} };
13
```

```
14      // Define a pair variable to receive the object that
15      // is returned from the equal_range member function.
16      pair<multimap<string, string>::iterator,
17           multimap<string, string>::iterator> range;
18
19      // Define an iterator for the multimap.
20      multimap<string, string>::iterator iter;
21
22      // Get the range of elements that match "Faye".
23      range = phonebook.equal_range("Faye");
24
25      // Display all of the elements that match "Faye".
26      for (iter = range.first; iter != range.second; iter++)
27      {
28         cout << iter->first << "\t" << iter->second << endl;
29      }
30
31      return 0;
32  }
```

**Program Output**

```
Faye    555-0707
Faye    555-1234
```

Let's take a closer look at the program:

- Lines 9 through 12 define a `multimap` container named `phonebook`, initialized with six elements.
- Lines 16 and 17 define a `pair` variable named `range`. The `range` variable will receive the object that is returned from the `equal_range` member function. The object's `first` and `second` members will both be iterators for the `multimap` container.
- Line 20 defines an iterator named `iter` for the `multimap` container.
- Line 23 calls the `phonebook` container's `equal_range` member function, specifying "Faye" as the key to search for. The `equal_range` function returns a `pair` object that is assigned to the `range` variable.
- The `for` loop in lines 26 through 29 iterates over all of the elements matching the key "Faye", displaying their values. Here are the details of the loop:
  - The `iter` iterator is initially set to `range.first`.
  - The loop repeats as long as `iter` is not equal to `range.second`.
  - At the end of each loop iteration, `iter` is incremented to the next element.
  - Each time the loop iterates, it displays the contents of the element to which `iter` is pointing (line 26).



**NOTE:** When searching for a range of elements with the `equal_range` member function, if the specified key is not found, both iterators will point to the element that would naturally appear after the element that was searched for.

## Deleting Elements from a multimap

You can delete, or erase, elements from a multiap using the erase() member function. You call the erase() member function, passing the key that is associated with the elements that you want to erase. All of the elements matching the specified key will be erased from the multimap. The erase() member function returns the number of elements that were erased. The following code shows an example:

```
// Define a phonebook multimap.
multimap<string, string> phonebook =
   { {"Will", "555-1212"},  {"Will", "555-0123"},
     {"Faye", "555-0707"},  {"Faye", "555-1234"},
     {"Sarah", "555-8787"}, {"Sarah", "555-5678"} };

// Delete Will's phone numbers from the multimap.
phonebook.erase("Will");
```

This code creates a multimap container with six elements. Then, it deletes all of the elements with the key "Will". (Two elements will be deleted.) The multimap then contains only four elements.

## The unordered_multimap Class



Beginning in C++11, the STL provides a class template named unordered_multimap that is similar to the multimap class, except in two regards: (1) the keys in an unordered_multimap are not sorted in any particular order, and (2) the unordered_multimap class has better performance. If you will be making a lot of searches on a large number of elements, and you are not concerned with retrieving them in key order, you should probably use the unordered_multimap class instead of the multimap class.

To use the unordered_multimap class, you must write the #include <unordered_multimap> directive in your program. The unordered_multimap class has a similar set of member functions as the multimap class, and in most cases, working with an unordered_multimap is just like working with a multimap.



**NOTE:** The differences between the unordered_multimap class and the multimap class are a result of the way each internally stores its elements. The multimap class uses a data structure known as a *binary tree*, and the unordered_multimap class uses a *hash table*.



### Checkpoint

17.20 Each element that is stored in a map has two parts. What are they?

17.21 Write a statement that defines a map named myMap. The keys in myMap should be ints, and the values should be strings.

17.22 Suppose an empty map named employee has been created. What does the following statement do?

```
employee[543] = "Joanne Manchester";
```

17.23 Describe two ways in which you can retrieve an element with a particular key in a map.

17.24 If you want to store objects of a class you have written, as values in a map, what do you have to make sure that the class has?

17.25 If you want to store objects of a class you have written, as keys in a map, what do you have to make sure that the class has?

17.26 What is the difference between a map and an unordered_map?

17.27 What is the difference between a map and a multimap?

## 17.5 The set, multiset, and unordered_set Classes

**CONCEPT: A set contains a collection of unique values.**


VideoNote
**The set Container**

A *set* is an associative container that stores a collection of unique values in a way that is similar to a mathematical set. You use the STL class template set to implement a set container in C++. Here are two important things to know about the STL set container:

- All the elements in a set must be unique. No two elements can have the same value.
- The elements in a set are automatically sorted in ascending order.

To use the set class, you will need to #include the <set> header file in your program. Then, you can define a set object using one of the set class constructors. Table 17-12 shows the general format of a set definition statement using each constructor. Table 17-13 lists many (but not all) of the set class's member functions.

**Table 17-12** set Definition Statements

| | |
|---|---|
| **Default Constructor** | `set<dataType> name;`<br>Creates an empty set object. In the general format, *dataType* is the data type of each element, and *name* is the name of the set. |
| **Range Constructor** | `set<dataType> name(iterator1, iterator2);`<br>Creates a set object that initially contains a range of values specified by two iterators. In the general format, *dataType* is the data type of each element, and *name* is the name of the set. The *iterator1* and *iterator2* arguments mark the beginning and end of a range of values that will be stored in the set. (*iterator2* marks the end of the range, but the element pointed to by *iterator2* will not be included in the range.) Any duplicate values in the range will be added only once to the set. |
| **Copy Constructor** | `set<dataType> name(set2);`<br>Creates a set object that is a copy of another set object. In the general format, *dataType* is the data type of each element, *name* is the name of the set, and *set2* is the set to copy. |

**Table 17-13** Some of the `set` Member Functions

| Member Function | Description |
|---|---|
| `begin()` | Returns an iterator to the first element in the container. |
| `cbegin()` | Returns a `const_iterator` to the first element in the container. |
| `cend()` | Returns a `const_iterator` pointing to the end of the container. |
| `clear()` | Erases all of the elements in the container. |
| `count(value)` | Returns the number of elements containing the specified value. (For the `set` class, the `count()` function returns either 0 or 1. For the `multiset` class, the function can return values greater than 1.) |
| `crbegin()` | Returns a `const_reverse_iterator` pointing to the last element in the container. |
| `crend()` | Returns a `const_reverse_iterator` pointing to the first element in the container. |
| `emplace(args...)` | Constructs a new element into the container, passing the list of arguments to the element's constructor. If an element with the specified value already exists, the function call does nothing. |
| `empty()` | Returns `true` if the container is empty, or `false` otherwise. |
| `end()` | Returns an iterator to the end of the container (the position after the last element). |
| `equal_range(value)` | Returns a `pair` object. The `pair` object's `first` member is an iterator pointing to the first element in the `set` that matches the specified *`value`*. The `pair` object's `second` member is an iterator pointing to the position *after* the last element that matches the specified *`value`*. If the specified *`value`* is not found, both iterators will point to the element that would naturally appear after the element that was searched for. (With the `set` class, the range will have at most one element. With the `multiset` class, the range can have multiple elements.) |
| `erase(value)` | Erases the element containing the specified value. The function returns 1 if the element was erased, or 0 if no matching element was found. |
| `find(value)` | Searches for an element with the specified value. If the element is found, the function returns an iterator to it. If the element is not found, the function returns an iterator to the end of the `set`. |
| `insert(value)` | Inserts a value as an element to the `set`. If an element with the specified value already exists, the function does not insert a new element. The function returns a `pair` object, with its `first` member being an iterator pointing to the newly inserted element (or the equivalent element, if it was already present), and with the `second` member being the `bool` value `true` if a new element was inserted, or `false` if the element was already present. |
| `lower_bound(value)` | Returns an iterator pointing to the first element with a key that is equal to or greater than *`value`*. |
| `max_size()` | Returns the theoretical maximum size of the container. |
| `rbegin()` | Returns a `reverse_iterator` pointing to the last element in the container. |
| `rend()` | Returns a `reverse_iterator` pointing to the first element in the container. |
| `size()` | Returns the number of elements in the container. |
| `swap(second)` | The *`second`* argument must be a `map` object of the same type as the calling object. The function swaps the contents of the calling object and the *`second`* object. |
| `upper_bound(value)` | Returns an iterator pointing to the first element with a key that is greater than *`value`*. |

Here is an example of how you might define a set container to hold integers:

```
set<int> numbers;
```

This statement defines a set container named numbers. Inside the angled brackets <>, the data type int is the type of each element. You can provide an initialization list to initialize the set, as shown here:

```
set<int> numbers = {1, 2, 3, 4, 5};
```

Here is an example of defining a set to hold strings:

```
set<string> names = {"Joe", "Karen", "Lisa", "Jackie"};
```

A set cannot contain duplicate items, so, if the same value appears more than once in an initialization list, it will be added to the set only one time. For example, the following set will contain the values 1, 2, 3, 4, and 5:

```
set<int> numbers = {1, 1, 2, 3, 3, 3, 4, 5, 5};
```

## Adding Elements to an Existing set

The set class provides an insert() member function that adds a new element to the container. You simply call the member function, passing the new element's value as an argument. The following code shows an example:

```
set<int> numbers;
numbers.insert(10);
numbers.insert(20);
numbers.insert(30);
```

The first statement creates an empty set container named numbers. The statements that follow add the values 10, 20, and 30 to the set. If the value that you are inserting already exists in the set, the insert()member function will do nothing (no new element will be inserted).



**NOTE:** The set class also provides the emplace() member function, for inserting elements. You will see an example of it momentarily. For a review of the difference between the emplace() and insert() member functions, see the discussion on emplacement that appears in this chapter's section on vectors.

## Iterating Over a set with the Range-Based for Loop

The range-based for loop is a convenient way to iterate over all of the elements in a set. The following code shows an example of how to display all of the elements in a set:

```
// Create a set containing names.
set<string> names = {"Joe", "Karen", "Lisa", "Jackie"};

// Display each element.
for (string element : names)
{
   cout << element << endl;
}
```

## Using an Iterator with a set

You can use a bidirectional iterator to access the elements of a set. The set class provides the begin() and cbegin() member functions that return an iterator pointing to the first element in the set, and the end() and cend() member functions that return an iterator pointing to the end of the set (the position *after* the last element). The following code shows an example of using an iterator to display all of the elements of a set:

```
// Create a set containing names.
set<string> names = {"Joe", "Karen", "Lisa", "Jackie"};

// Create an iterator.
set<string>::iterator iter;

// Use the iterator to display each element in the set.
for (iter = names.begin(); iter != names.end(); iter++)
{
   cout << *iter << endl;
}
```



**NOTE:** The end() and cend() member functions return an iterator pointing to the end of the set, but it does not point to an actual element. It points to the position where an additional element would exist, if it appeared after the last element.

## Determining Whether a Value Exists in a set

You can use either the count() member function or the find() member function to determine whether a value exists in a set. The count() member function accepts the desired value as an argument, and returns 1 if the value is found in the set, or 0 if the value is not found. Here is an example:

```
set<string> names = {"Joe", "Karen", "Lisa", "Jackie"};
if (names.count("Lisa"))
   cout << "Lisa was found in the set.\n";
else
   cout << "Lisa was not found.\n";
```

The find() member function accepts the desired value as an argument, and returns an iterator that points to the element if it is found in the set. If the element is not found, the find() function returns an iterator to the end of the set. The following code shows an example:

```
// Create a set containing names.
set<string> names = {"Joe", "Karen", "Lisa", "Jackie"};

// Create an iterator.
set<string>::iterator iter;

// Find "Karen".
iter = names.find("Karen");

// Display the result.
if (iter != names.end())
```

```
{
   cout << *iter << " was found.\n";
}
else
{
   cout << "Karen was not found.\n";
}
```

## Storing Objects of Your Own Classes in a set

You can store objects of a class that you have written in a set, as long as the class has overloaded the < operator. The set class uses the < operator to determine the order in which to store elements, and to detect duplicates. (Suppose we are comparing two values: value1 and value2. If value1 is *NOT* less than value2, and value2 is *NOT* less than value1, then the two values are equal. This is the logic that the set class uses to determine whether two values are duplicates.)

For example, recall the Customer class shown earlier in this chapter, in our discussion of the map class. (See the contents of the Customer.h file.) The Customer class holds two pieces of data about a customer: the customer number and the customer's name. The class also has an overloaded < operator that compares customer numbers. If we store objects of the Customer class in a set container, the objects will be sorted by customer number, and the container will not allow objects with the same customer number to be stored. Program 17-22 demonstrates this.

**Program 17-22**

```
1  #include <iostream>
2  #include <set>
3  #include "Customer.h"
4  using namespace std;
5
6  int main()
7  {
8     // Create a set of Customer objects.
9     set<Customer> customerset =
10      { Customer(1003, "Megan Cruz"),
11        Customer(1002, "Austin Hill"),
12        Customer(1001, "Sarah Scott")
13      };
14
15    // Try to insert a duplicate customer number.
16    customerset.emplace(1001, "Evan Smith");
17
18    // Display the set elements
19    cout << "List of customers:\n";
20    for (auto element : customerset)
21    {
22       cout << element.getCustNumber() << " "
23            << element.getName() << endl;
24    }
25
```

*(program continues)*

**Program 17-22** *(continued)*

```
26    // Search for customer number 1002.
27    cout << "\nSearching for Customer Number 1002:\n";
28    auto it = customerset.find(Customer(1002, ""));
29
30    if (it != customerset.end())
31       cout << "Found: " << it->getName() << endl;
32    else
33       cout << "Not found.\n";
34
35    return 0;
36 }
```

**Program Output**

```
List of customers:
1001 Sarah Scott
1002 Austin Hill
1003 Megan Cruz

Searching for Customer Number 1002:
Found: Austin Hill
```

Let's take a closer look at the program:

- Lines 9 through 13 define a `set` container named `customerset`. The container is initialized with three `Customer` objects.
- Line 16 calls the `emplace()` member function to add a `Customer` object to the `set`. However, the customer number that is being passed to the `emplace()` function is already in the `set`. As a result, the object will not be added to the `set`.
- The loop in lines 20 through 24 displays all of the `set` container's elements. Notice in the program output, the elements are retrieved in order of their customer numbers.
- Next, the program searches the `set` container for an object with the customer number 1002. The statement in line 28 defines an iterator, assigning it the value returned from the `find()` member function.

  Look carefully at the argument we are passing to the `find()` member function. The expression `Customer(1002, "")` constructs a temporary, nameless `Customer` object that has 1002 as the customer number, and `""` as the customer name. (Because we are simply using this object to search for an element with customer number 1002, there is no need to provide a meaningful value for the customer name.) The `find()` member function will accept this `Customer` object as an argument, and it will search the container for an element that has a matching customer number. (Remember, the `set` container uses the `Customer` class's overloaded `<` operator to determine whether two objects are equal.) If a matching element is found, the function returns an iterator pointing to it. Otherwise, it returns an iterator pointing to the end of the container.

- The if/else statement in lines 30 through 33 tests the iterator to determine whether it is pointing to a matching element, or the end of the container, and displays a message indicating the results of the search.

## The multiset Class

The multiset class lets you create a set container that can store duplicate elements. To use the multiset class, you will need to #include the <set> header file in your program. The class provides the same member functions as the set class, with these two differences:

- In the set class, the count() member function returns either 0 or 1. In the multiset class, the count() member function can return values greater than 1.
- In the set class, the equal_range() member function returns a range with, at most, one element. In the multiset class, the equal_range() member function can return a range with multiple elements.

## The unordered_set and unordered_multiset Classes



Beginning in C++11, the STL provides class templates named unordered_set and unordered_multiset. These are similar to the set and multiset classes, except in two regards: (1) the values stored in an unordered_set or an unordered_multiset are not sorted in any particular way, and (2) the unordered_set and unordered_multiset classes has better performance than the set and multiset classes. If you will be making a lot of searches on a large number of elements, and you are not concerned with retrieving them in any order, you should probably use the unordered_set or unordered_multiset classes instead of set or multiset.

To use the unordered_set or unordered_multiset classes, you must write the #include <unordered_set> directive in your program. In most cases, working with an unordered_set or unordered_multiset is similar to working with a set or multiset.



### Checkpoint

17.28 What are two differences between a set and a vector?

17.29 Write a statement that defines an empty set object named aset, that can hold strings.

17.30 Write a statement that defines a set object named aset, that can hold ints. The set should be initialized with these values: 10, 20, 30, 40

17.31 What happens when you use the insert() member function to insert a value into a set, and that value is already in the set?

17.32 What value does the set class's count member function return?

17.33 If you store objects of a class that you have written in a set, what must the class overload?

17.34 What is the difference between a set and a multiset?

17.35 In what two ways are the unordered_set and unordered_multiset different from the set and multiset classes?

## 17.6 Algorithms

**CONCEPT: Many commonly used algorithms are written as function templates in the STL.**

The STL provides a number of algorithms, implemented as function templates, in the `<algorithm>` header file. These functions perform various operations on ranges of elements. A *range of elements* is a sequence of elements denoted by two iterators. The first iterator points to the first element in the range, and the second iterator points to the end of the range (the element to which the second iterator points is not included in the range). The algorithms can be organized in the following categories:

- **Min/max algorithms:**
  Determine the smallest (min) and largest (max) values in a range of elements
- **Sorting algorithms:**
  Sort a range of elements, or determine whether a range is sorted
- **Search algorithms:**
  Perform various searching operations on a sorted range of elements
- **Read-only sequence algorithms:**
  Iterate over a range of elements, performing various operations that do not modify the elements
- **Copying and moving algorithms:**
  Use various techniques to copy or move ranges of elements.
- **Swapping algorithms:**
  Swap values, ranges of elements, or the values pointed to by iterators
- **Replacement algorithms:**
  Replace elements of a specified value
- **Removal algorithms:**
  Remove elements
- **Reversal algorithms:**
  Reverse the order of elements in a range
- **Fill algorithms:**
  Fill the elements in a range with values
- **Rotation algorithms:**
  Rotate the elements in a range with values
- **Shuffling algorithms:**
  Shuffle the elements in a range
- **Set algorithms:**
  Perform common mathematical set operations on a range of elements
- **Transformation algorithm:**
  Iterate over ranges of elements, performing an operation using each element
- **Partition algorithms:**
  Partition a range of elements into two groups
- **Merge algorithms:**
  Merge ranges of elements

- **Permutation algorithms:**
  Rearrange a range of elements into all of its different possible permutations
- **Heap algorithms:**
  Create and work with a heap data structure
- **Lexicographical comparison algorithm:**
  Makes a lexicographical comparison between two ranges of elements

At the time this was written, there are 85 function templates in the `<algorithm>` header file. For a summary of all of the STL algorithms, see Appendix H on the Computer Science Portal at www.pearsonglobaleditions.com/gaddis. We will not discuss all of the STL algorithms in this chapter, but we will look at examples of some of the more interesting ones.

## Sorting and Searching Algorithms

The `<algorithm>` header file defines several function templates for sorting and searching. We will look at two of them: `sort()` and `binary_search()`. The `sort` function takes the following general format:

```
sort(iterator1, iterator2)
```

In the general format, *`iterator1`* and *`iterator2`* mark the beginning and end of a range of elements. The function sorts the range of elements in ascending order. The `binary_search()` function takes the following general format:

```
binary_search(iterator1, iterator2, value)
```

In the general format, the *`iterator1`* and *`iterator2`* arguments mark the beginning and end of a range of elements that are sorted in ascending order, and *`value`* is the value to search for. The function returns `true` if the *`value`* is found in the range of elements, or `false` otherwise. Program 17-23 demonstrates both the `sort()` and `binary_search()` functions.

**Program 17-23**

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5
6  int main()
7  {
8     int searchValue;    // Value to search for
9
10    // Create a vector of unsorted integers.
11    vector<int> numbers = {10, 1, 9, 2, 8, 3, 7, 4, 6, 5};
12
13    // Sort the vector.
14    sort(numbers.begin(), numbers.end());
15
```

*(program continues)*

**Program 17-23** *(continued)*

```
16      // Display the vector.
17      cout << "Here are the sorted values:\n";
18      for (auto element : numbers)
19         cout << element << " ";
20      cout << endl;
21
22      // Get the value to search for.
23      cout << "Enter a value to search for: ";
24      cin >> searchValue;
25
26      // Search for the value.
27      if (binary_search(numbers.begin(), numbers.end(), searchValue))
28         cout << "That value is in the vector.\n";
29      else
30         cout << "That value is not in the vector.\n";
31
32      return 0;
33   }
```

**Program Output**

```
Here are the sorted values:
1 2 3 4 5 6 7 8 9 10
Enter a value to search for: 8
That value is in the vector.
```

**Program Output**

```
Here are the sorted values:
1 2 3 4 5 6 7 8 9 10
Enter a value to search for: 99
That value is not in the vector.
```

The `sort()` and `binary_search()` functions use the `<` operator to compare elements. If the elements that you are sorting and searching contain your own class objects, be sure to overload the `<` operator. For example, recall the `Customer` class shown earlier in this chapter, in our discussion of the `map` class. (See the contents of the Customer.h file.) The `Customer` class holds two pieces of data about a customer: the customer number and the customer's name. The class also has an overloaded `<` operator that compares customer numbers. If we store objects of the `Customer` class in a sequence container, we can use the STL `sort()` and `binary_search()` functions to sort and search the container for an object containing a particular customer number. Program 17-24 demonstrates this.

**Program 17-24**

```
1   #include <iostream>
2   #include <vector>
3   #include <algorithm>
4   #include "Customer.h"
5   using namespace std;
6
```

```
 7  int main()
 8  {
 9     int searchValue;    // Value to search for
10
11     // Create a vector of unsorted Customer objects.
12     vector<Customer> customers =
13        { Customer(1003, "Megan Cruz"),
14          Customer(1001, "Sarah Scott"),
15          Customer(1002, "Austin Hill")
16        };
17
18     // Sort the vector.
19     sort(customers.begin(), customers.end());
20
21     // Display the vector.
22     cout << "Here are the sorted customers:\n";
23     for (auto element : customers)
24     {
25        cout << element.getCustNumber() << " "
26             << element.getName() << endl;
27     }
28     cout << endl;
29
30     // Get the customer number to search for.
31     cout << "Enter a customer number to search for: ";
32     cin >> searchValue;
33
34     // Search for the customer number.
35     if (binary_search(customers.begin(), customers.end(),
36                       Customer(searchValue, "")))
37        cout << "That customer is in the vector.\n";
38     else
39        cout << "That customer is not in the vector.\n";
40
41     return 0;
42  }
```

**Program Output**

```
Here are the sorted customers:
1001 Sarah Scott
1002 Austin Hill
1003 Megan Cruz

Enter a customer number to search for: 1001
That customer is in the vector.
```

**Program Output**

```
Here are the sorted customers:
1001 Sarah Scott
1002 Austin Hill
1003 Megan Cruz

Enter a customer number to search for: 1009
That customer is not in the vector.
```

Let's take a closer look at the program:

- Lines 12 through 16 define a `vector` named `customers`, initialized with three `Customer` objects.
- Line 19 calls the `sort` function to sort the `vector`. Because the `Customer` class's overloaded `<` operator compares customer numbers, the objects in the `vector` will be sorted by customer number, in ascending order.
- The loop in lines 23 through 27 displays all of the `vector`'s elements.
- Line 31 prompts the user to enter a customer number, and line 32 stores the user's input in the `searchValue` variable.
- Next, the program searches the `vector` for an object with the customer number entered by the user. Look carefully at the third argument we are passing to the `binary_search()` function. The expression `Customer(searchValue, "")` constructs a temporary, nameless `Customer` object that has `searchValue`'s value as the customer number, and `""` as the customer name. (Because we are simply using this object to search for an element with a specified customer number, there is no need to provide a meaningful value for the customer name.) The `binary_search()` function will accept this `Customer` object as an argument, and it will search the specified range of elements for an object that has a matching customer number. (The `binary_search` function will use the `Customer` class's overloaded `<` operator to determine whether two objects are equal.) If a matching element is found, the function returns `true`. Otherwise, it returns `false`.

## Detecting Permutations

If a range has *N* elements, there are *N*! possible arrangements of those elements. Each arrangement is called a *permutation*. For example, suppose we have a vector with the following integer elements:

```
1, 2, 3
```

There are three elements in the vector, so there are six possible ways to rearrange the elements. The six possible permutations are:

```
1, 2, 3
1, 3, 2
2, 1, 3
2, 3, 1
3, 1, 2
3, 2, 1
```

The STL's `is_permutation()` function determines whether one range of elements is a permutation of another range of elements. Here is the function's general format:

```
is_permutation(iterator1, iterator2, iterator3)
```

In the general format, *`iterator1`* and *`iterator2`* mark the beginning and end of the first range of elements. The *`iterator3`* argument marks the beginning of the second range of elements, assumed to have the same number of elements as the first range. The function returns `true` if the second range is a permutation of the first range, or `false` otherwise.

Program 17-25 demonstrates the `is_permutation()` function. It uses the function to determine whether a set of numbers matches the winning lottery numbers.

**Program 17-25**

```
 1  #include <iostream>
 2  #include <vector>
 3  #include <algorithm>
 4  using namespace std;
 5
 6  int main()
 7  {
 8     const int MAX = 5;           // Numbers in 1 lottery ticket
 9     vector<int> winning(MAX);    // The winning numbers
10     vector<int> player(MAX);     // Numbers on a ticket
11
12     // Get the winning numbers.
13     cout << "Enter the " << MAX << " winning numbers:\n";
14     for (auto &element : winning)
15     {
16        cout << "> ";
17        cin >> element;
18     }
19
20     // Get the numbers purchased on a lottery ticket.
21     cout << "\nEnter your " << MAX << " lottery numbers:\n";
22     for (auto &element : player)
23     {
24        cout << "> ";
25        cin >> element;
26     }
27
28     // Check for a winner.
29     if (is_permutation(winning.begin(), winning.end(),
30                        player.begin()))
31        cout << "You won the lottery!\n";
32     else
33        cout << "Sorry, you did not win.\n";
34
35     return 0;
36  }
```

**Program Output**

```
Enter the 5 winning numbers:
> 10
> 25
> 67
> 88
> 93

Enter your 5 lottery numbers:
> 25
> 15
> 62
```

*(program output continues)*

**Program 17-25** *(continued)*

```
> 93
> 88
Sorry, you did not win.
```

**Program Output**

```
Enter the 5 winning numbers:
> 10
> 25
> 67
> 88
> 93

Enter your 5 lottery numbers:
> 67
> 10
> 93
> 88
> 25
You won the lottery!
```

## Plugging Your Own Functions into an Algorithm

When a C++ program is running, the executable code for each of the functions in that program is stored in memory. You can use the name of a function to get that function's address in memory, in the same way that you can use the name of an array to get that array's address in memory. This capability allows you to get a *function pointer*, which is a pointer to a function's executable code.

Many of the function templates in the STL are designed to accept function pointers as arguments. This allows you to "plug" one of your own functions into the algorithm. For example, here is the general format of the `for_each` function:

```
for_each(iterator1, iterator2, function)
```

In the general format, the *`iterator1`* and *`iterator2`* arguments mark the beginning and end of a range of elements. The *`function`* argument is the address of a function that accepts an element as its argument (Any value returned from *`function`* is ignored). The `for_each()` function iterates over the range of elements, passing each element as an argument to *`function`*.

Program 17-26 demonstrates the `for_each()` function. The program defines a function named `doubleNumber()`. The `doubleNumber()` function accepts a reference to an `int` as its argument, and it doubles the value of the argument. The program uses the `for_each()` function to double the value of every element in a `vector` of integers.

**Program 17-26**

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5
```

```
 6  // Function prototype
 7  void doubleNumber(int &);
 8
 9  int main()
10  {
11     vector<int> numbers = { 1, 2, 3, 4, 5 };
12
13     // Display the numbers before doubling.
14     for (auto element : numbers)
15        cout << element << " ";
16     cout << endl;
17
18     // Double the value of each vector element.
19     for_each(numbers.begin(), numbers.end(), doubleNumber);
20
21     // Display the numbers before doubling.
22     for (auto element : numbers)
23        cout << element << " ";
24     cout << endl;
25
26     return 0;
27  }
28
29  //*****************************************************
30  // The doubleNumber function doubles the value of n. *
31  //*****************************************************
32  void doubleNumber(int &n)
33  {
34     n = n * 2;
35  }
```

**Program Output**

```
1 2 3 4 5
2 4 6 8 10
```

Another example is the `count_if()` function. The general format of the `count_if()` function is:

```
count_if(iterator1, iterator2, function)
```

In the general format, the *`iterator1`* and *`iterator2`* arguments mark the beginning and end of a range of elements. The *`function`* argument is the address of a function that accepts an element as its argument, and returns either `true` or `false`. (The function should not make changes to the element.) The `count_if` function iterates over the range of elements, passing each element as an argument to *`function`*. The `count_if` function returns the number of elements for which *`function`* returns `true`.

Program 17-27 demonstrates the `count_if()` function. The program defines a function named `outOfRange()`. The `outOfRange()` function accepts an `int` as its argument, and it returns `true` if the argument is less than 0 or greater than 100. The program uses the `count_if()` function to determine the number of elements in a `vector` that are not within the range of 1 through 100.

**Program 17-27**

```
 1  #include <iostream>
 2  #include <vector>
 3  #include <algorithm>
 4  using namespace std;
 5
 6  // Function prototype
 7  bool outOfRange(int);
 8
 9  int main()
10  {
11     // Create a vector of ints.
12     vector<int> numbers = { 0, 99, 120, -33, 10, 8, -1, 101 };
13
14     // Get the number of elements that are < 0 or > 100.
15     int invalid = count_if(numbers.begin(), numbers.end(), outOfRange);
16
17     // Display the results.
18     cout << "There are " << invalid << " elements out of range.\n";
19     return 0;
20  }
21
22  //*************************************************************
23  // The outOfRange function returns true if n is out of range. *
24  //*************************************************************
25  bool outOfRange(int n)
26  {
27     // Constants for min and max values
28     const int MIN = 0, MAX = 100;
29
30     // Flag to hold the status
31     bool status;
32
33     // Determine whether n out of range.
34     if (n < MIN || n > MAX)
35        status = true;
36     else
37        status = false;
38
39     return status;
40  }
```

**Program Output**

```
There are 4 elements out of range.
```

## Using the STL to Perform Set Operations

The STL provides a number of function templates for performing the basic set operations you probably learned in math class. Table 17-14 summarizes these methods.

**Table 17-14** STL Algorithms to Perform Set Operations

| Function Template | Description |
|---|---|
| `set_union(`*`iterator1`*`, `*`iterator2`*`, `*`iterator3`*`, `*`iterator4`*`, `*`iterator5`*`)` | Finds the union of two sets. The union of two sets is a set that contains all the elements of both sets, excluding duplicates.<br>The *`iterator1`* and *`iterator2`* arguments mark the beginning and end of the first set. The *`iterator3`* and *`iterator4`* arguments mark the beginning and end of the second set. The *`iterator5`* argument marks the beginning of the container that will hold the union of the two sets.<br>The function returns an iterator pointing to the end of the range of elements in the union. |
| `set_intersection (`*`iterator1`*`, `*`iterator2`*`, `*`iterator3`*`, `*`iterator4`*`, `*`iterator5`*`)` | Finds the intersection of two sets. The intersection of two sets is a set that contains only the elements that are found in both sets.<br>The *`iterator1`* and *`iterator2`* arguments mark the beginning and end of the first set. The *`iterator3`* and *`iterator4`* arguments mark the beginning and end of the second set. The *`iterator5`* argument marks the beginning of the container that will hold the intersection of the two sets.<br>The function returns an iterator pointing to the end of range of elements in the intersection. |
| `set_difference (`*`iterator1`*`, `*`iterator2`*`, `*`iterator3`*`, `*`iterator4`*`, `*`iterator5`*`)` | Finds the difference of two sets. The difference of two sets is the set of elements that appear in one set, but not the other.<br>The *`iterator1`* and *`iterator2`* arguments mark the beginning and end of the first set. The *`iterator3`* and *`iterator4`* arguments mark the beginning and end of the second set. The *`iterator5`* argument marks the beginning of the container that will hold the difference of the two sets.<br>The function returns an iterator pointing to the end of the range of elements in the difference. |
| `set_symmetric_difference (`*`iterator1`*`, `*`iterator2`*`, `*`iterator3`*`, `*`iterator4`*`, `*`iterator5`*`)` | Finds the symmetric difference of two sets. The symmetric difference of two sets is the set of elements that are in one set, but not in both.<br>The *`iterator1`* and *`iterator2`* arguments mark the beginning and end of the first set. The *`iterator3`* and *`iterator4`* arguments mark the beginning and end of the second set. The *`iterator5`* argument marks the beginning of the container that will hold the symmetric difference of the two sets.<br>The function returns an iterator pointing to the end of the range of elements in the symmetric difference. |
| `includes(`*`iterator1`*`, `*`iterator2`*`, `*`iterator3`*`, `*`iterator4`*`)` | Determines whether one set includes another set.<br>The *`iterator1`* and *`iterator2`* arguments mark the beginning and end of the first set. The *`iterator3`* and *`iterator4`* arguments mark the beginning and end of the second set.<br>The function returns `true` if the first set contains all of the elements of the second set. Otherwise, the function returns `false`. |

The ranges of elements with which these member functions work can be stored in `set` containers, `vectors`, `arrays`, or any other type of container that supports iterators. Keep in mind, however, the ranges of elements must be sorted in ascending order before you can use any of the functions in Table 17-14 with them. For that reason, the `set` container is convenient because it automatically sorts its elements.

### Finding the Union of Sets with the `set_union` Function

The union of two sets is a set that contains all the elements of both sets, with no duplicates. For example, suppose `set1` contains the values 1, 2, 3, 4, and `set2` contains the values 3, 4, 5, 6. The union of `set1` and `set2` would be a set containing the values 1, 2, 3, 4, 5, 6.

You can call the STL algorithm function `set_union` to get the union of two sets. The function stores the union in a third container. Program 17-28 demonstrates how to use the function.

**Program 17-28**

```
#include <iostream>
#include <set>
#include <algorithm>
#include <vector>
using namespace std;

int main()
{
   // Create two sets.
   set<int> set1 = {1, 2, 3, 4};
   set<int> set2 = {3, 4, 5, 6};

   // Create a vector to hold the union. The
   // vector must be large enough to hold both sets.
   vector<int> result(set1.size() + set2.size());

   // Get the union of the sets. The result vector
   // will hold the union, and iter will point to
   // the end of the result vector.
   auto iter = set_union(set1.begin(), set1.end(),
                         set2.begin(), set2.end(),
                         result.begin());

   // Resize the result vector to remove unused elements.
   result.resize(iter - result.begin());

   // Display the result vector's elements
   cout << "The union of the sets is:\n";
   for (auto element : result)
   {
      cout << element << " ";
   }
```

```
33      cout << endl;
34
35      return 0;
36  }
```

**Program Output**

```
The union of the sets is:
1 2 3 4 5 6
```

Let's take a closer look at Program 17-28:

- Lines 10 and 11 define `set1` and `set2`, initialized with the values 1, 2, 3, 4, and 3, 4, 5, 6, respectively.
- Line 15 defines a `vector` named `result`. We will use the `vector` to hold the union of `set1` and `set2`. Notice the value we are passing to the `vector` constructor: `set1.size() + set2.size()`. This causes the `vector` to be large enough to hold both sets. (The `vector` elements will be initialized with 0.) Figure 17-6 shows the state of `set1`, `set2`, and `result` at this point in the program.
- Line 18 defines an iterator that can be used with the `vector`. The name of the iterator is `iter`.
- Lines 20 through 22 call the `set_union` function. The function gets the union of `set1` and `set2`, and stores the union in the `result` vector. The function returns an iterator that is assigned to `iter`. The `iter` iterator points to the *end* of the range of elements in the union. Figure 17-7 shows the state of `set1`, `set2`, `result`, and `iter` at this point in the program. (Notice the last two elements of the `result` vector are unused.)
- Line 25 resizes the `result` vector to remove the unused elements. Figure 17-8 shows the state of `set1`, `set2`, `result`, and `iter` at this point in the program.
- Lines 28 through 36 display the program's output, which includes the contents of the `result` vector.

**Figure 17-6** State of `set1`, `set2`, and `result` after line 15 has executed

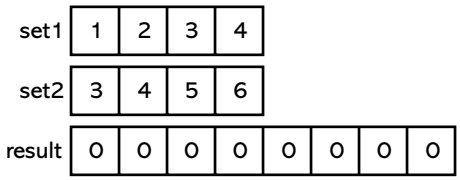



**Figure 17-7** State of `set1`, `set2`, `result`, and `iter` after lines 20 through 22 have executed

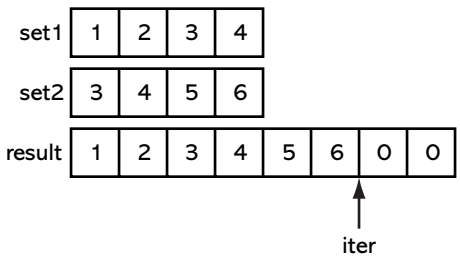
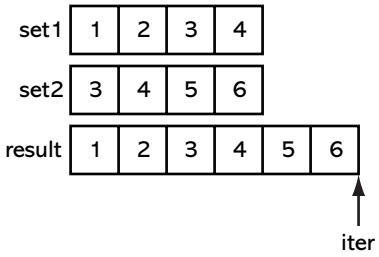


**Figure 17-8** State of `set1`, `set2`, `result`, and `iter` after line 25 has executed




### Finding the Intersection of Sets with the `set_intersection` Function

The intersection of two sets is a set that contains only the elements that are found in both sets. For example, suppose `set1` contains the values 1, 2, 3, 4, and `set2` contains the values 3, 4, 5, 6. The intersection of `set1` and `set2` would be a set containing the values 3, 4.

You can call the STL algorithm function `set_intersection` to get the intersection of two sets. The function stores the intersection in a third container. Program 17-29 demonstrates how to use the function.

**Program 17-29**

```
 1  #include <iostream>
 2  #include <set>
 3  #include <algorithm>
 4  #include <vector>
 5  using namespace std;
 6
 7  int main()
 8  {
 9     // Create two sets.
10     set<int> set1 = {1, 2, 3, 4};
11     set<int> set2 = {3, 4, 5, 6};
12
13     // Create a vector to hold the intersection. The
14     // vector must be large enough to hold both sets.
15     vector<int> result(set1.size() + set2.size());
16
17     // Get the intersection of the sets. The result vector
18     // will hold the intersection, and iter will point to
19     // the end of the result vector.
20     auto iter = set_intersection(set1.begin(), set1.end(),
21                                  set2.begin(), set2.end(),
22                                  result.begin());
23
24     // Resize the result vector to remove unused elements.
25     result.resize(iter - result.begin());
26
27     // Display the result vector's elements
28     cout << "The intersection of the sets is:\n";
29     for (auto element : result)
```

```
30      {
31          cout << element << " ";
32      }
33      cout << endl;
34
35      return 0;
36  }
```

**Program Output**

```
The intersection of the sets is:
3 4
```

Let's take a closer look at Program 17-29:

- Lines 10 and 11 define `set1` and `set2`, initialized with the values 1, 2, 3, 4, and 3, 4, 5, 6, respectively.
- Line 15 defines a `vector` named `result`. We will use the `vector` to hold the intersection of `set1` and `set2`. Notice the value we are passing to the `vector` constructor: `set1.size() + set2.size()`. This causes the `vector` to be large enough to hold both `sets`. (The `vector` elements will be initialized with 0.)
- Line 18 defines an iterator that can be used with the `vector`. The name of the iterator is `iter`.
- Lines 20 through 22 call the `set_intersection` function. The function gets the intersection of `set1` and `set2`, and stores the intersection in the `result` vector. The function returns an iterator that is assigned to `iter`. The `iter` iterator points to the *end* of the range of elements in the intersection.
- Line 25 resizes the `result` vector to remove any unused elements.
- Lines 28 through 33 display the program's output, which includes the contents of the `result` vector.

### Finding the Difference of Sets with the `set_difference` Function

The difference of two sets is the set of elements that appear in the first set, but not the second set. For example, suppose `set1` contains the values 1, 2, 3, 4, and `set2` contains the values 3, 4, 5, 6. The difference of `set1` and `set2` would be a set containing the values 1, 2.

You can call the STL algorithm function `set_difference` to get the difference of two sets. The function stores the difference in a third container. Program 17-30 demonstrates how to use the function.

**Program 17-30**

```
1  #include <iostream>
2  #include <set>
3  #include <algorithm>
4  #include <vector>
5  using namespace std;
6
7  int main()
```

*(program continues)*

**Program 17-30** *(continued)*

```
 8  {
 9     // Create two sets.
10     set<int> set1 = {1, 2, 3, 4};
11     set<int> set2 = {3, 4, 5, 6};
12
13     // Create a vector to hold the difference. The
14     // vector must be large enough to hold both sets.
15     vector<int> result(set1.size() + set2.size());
16
17     // Get the difference of the sets. The result vector
18     // will hold the difference, and iter will point to
19     // the end of the result vector.
20     auto iter = set_difference(set1.begin(), set1.end(),
21                                set2.begin(), set2.end(),
22                                result.begin());
23
24     // Resize the result vector to remove unused elements.
25     result.resize(iter - result.begin());
26
27     // Display the result vector's elements
28     cout << "The difference of set1 and set2 is:\n";
29     for (auto element : result)
30     {
31        cout << element << " ";
32     }
33     cout << endl;
34
35     return 0;
36  }
```

**Program Output**

```
The difference of set1 and set2 is:
1 2
```

Let's take a closer look at Program 17-30:

- Lines 10 and 11 define `set1` and `set2`, initialized with the values 1, 2, 3, 4, and 3, 4, 5, 6, respectively.
- Line 15 defines a `vector` named `result`. We will use the `vector` to hold the difference of `set1` and `set2`. Notice the value we are passing to the `vector` constructor: `set1.size() + set2.size()`. This causes the `vector` to be large enough to hold both `sets`. (The `vector` elements will be initialized with 0.)
- Line 18 defines an iterator that can be used with the `vector`. The name of the iterator is `iter`.
- Lines 20 through 22 call the `set_difference` function. The function gets the difference of `set1` and `set2`, and stores the difference in the `result` vector. The function returns an iterator that is assigned to `iter`. The `iter` iterator points to the *end* of the range of elements in the difference.
- Line 25 resizes the `result` vector to remove any unused elements.
- Lines 28 through 33 display the program's output, which includes the contents of the `result` vector.

### Finding the Symmetric Difference of Sets with the `set_symmetric_difference` Function

The symmetric difference of two sets is the set of elements that are in either set, but not in both. For example, suppose `set1` contains the values 1, 2, 3, 4, and `set2` contains the values 3, 4, 5, 6. The symmetric difference of `set1` and `set2` would be a set containing the values 1, 2, 5, 6.

You can call the STL algorithm function `set_symmetric_difference` to get the symmetric difference of two sets. The function stores the symmetric difference in a third container. Program 17-31 demonstrates how to use the function.

**Program 17-31**

```
#include <iostream>
#include <set>
#include <algorithm>
#include <vector>
using namespace std;

int main()
{
   // Create two sets.
   set<int> set1 = {1, 2, 3, 4};
   set<int> set2 = {3, 4, 5, 6};

   // Create a vector to hold the symmetric difference.
   // The vector must be large enough to hold both sets.
   vector<int> result(set1.size() + set2.size());

   // Get the symmetric difference of the sets. The result
   // vector will hold the symmetric difference, and iter
   // will point to the end of the result vector.
   auto iter = set_symmetric_difference(set1.begin(), set1.end(),
                                        set2.begin(), set2.end(),
                                        result.begin());

   // Resize the result vector to remove unused elements.
   result.resize(iter - result.begin());

   // Display the result vector's elements
   cout << "The symmetric difference of the sets is:\n";
   for (auto element : result)
   {
      cout << element << " ";
   }
   cout << endl;

   return 0;
}
```

**Program Output**

```
The symmetric difference of the sets is:
1 2 5 6
```

Let's take a closer look at Program 17-31:

- Lines 10 and 11 define `set1` and `set2`, initialized with the values 1, 2, 3, 4, and 3, 4, 5, 6, respectively.
- Line 15 defines a `vector` named `result`. We will use the `vector` to hold the difference of `set1` and `set2`. Notice the value we are passing to the `vector` constructor: `set1.size() + set2.size()`. This causes the `vector` to be large enough to hold both `sets`. (The `vector` elements will be initialized with 0.)
- Line 18 defines an iterator that can be used with the `vector`. The name of the iterator is `iter`.
- Lines 20 through 22 call the `set_symmetric_difference` function. The function gets the symmetric difference of `set1` and `set2`, and stores the symmetric difference in the `result` vector. The function returns an iterator that is assigned to `iter`. The `iter` iterator points to the *end* of the range of elements in the symmetric difference.
- Line 25 resizes the `result` vector to remove any unused elements.
- Lines 28 through 33 display the program's output, which includes the contents of the `result` vector.

### Finding Subsets

Suppose you have two sets and one of those sets includes all of the elements of the other set. For example, suppose `set1` contains the values 1, 2, 3, 4, and `set2` contains the values 2, 3. In this example, `set1` contains all of the elements of `set2`, which means that `set2` is a *subset* of `set1`.

You can call the STL function `includes` to determine whether one set includes all of the elements of another set. You use iterators to pass two sorted ranges of elements to the `includes` function, and the function returns `true` if the first range includes all of the elements in the second range. Otherwise, the function returns `false`. Program 17-32 demonstrates how to use the function. The program determines whether the elements in `set1` include all of the elements in `set2`.

**Program 17-32**

```
 1  #include <iostream>
 2  #include <set>
 3  #include <algorithm>
 4  using namespace std;
 5
 6  int main()
 7  {
 8     // Create two sets.
 9     set<int> set1 = {1, 2, 3, 4};
10     set<int> set2 = {2, 3};
11
12     // Determine whether set1 includes the
13     // elements of set2.
14     if (includes(set1.begin(), set1.end(),
15                  set2.begin(), set2.end()))
16     {
17        cout << "set2 is a subset of set1.\n";
18     }
```

```
19      else
20      {
21         cout << "set2 is NOT a subset of set1.\n";
22      }
23
24      return 0;
25   }
```

**Program Output**

```
set2 is a subset of set1.
```



## In the Spotlight:

### Set Operations

In this section, you will look at Program 17-33, which demonstrates various set operations. The program creates two sets: one that holds the names of students on the baseball team, and another that holds the names of students on the basketball team. The program then performs the following operations:

- It finds the intersection of the sets to display the names of students who play both sports.
- It finds the union of the sets to display the names of students who play either sport.
- It finds the difference of the `baseball` and `basketball` sets to display the names of students who play baseball but not basketball.
- It finds the difference of the `basketball` and `baseball` (`basketball-baseball`) sets to display the names of students who play basketball but not baseball. It also finds the difference of the `baseball` and `basketball` (`baseball-basketball`) sets to display the names of students who play baseball but not basketball.
- It finds the symmetric difference of the `basketball` and `baseball` sets to display the names of students who play one sport but not both.

**Program 17-33**

```
1   #include <iostream>
2   #include <string>
3   #include <algorithm>
4   #include <set>
5   #include <vector>
6   using namespace std;
7
8   // Function prototypes
9   void displaySet(set<string>);
10  void displayIntersection(set<string>, set<string>);
11  void displayUnion(set<string>, set<string>);
12  void displayDifference(set<string>, set<string>);
13  void displaySymmetricDifference(set<string>, set<string>);
14
```

*(program continues)*

**Program 17-33** *(continued)*

```
15  int main()
16  {
17       // Create sets for the baseball & basketball teams.
18       set<string> baseball = {"Jodi", "Carmen", "Aida", "Alicia"};
19       set<string> basketball = {"Eva", "Carmen", "Alicia", "Sarah"};
20
21       // Display the elements of the baseball set.
22       cout << "The following students are on the baseball team:\n";
23       displaySet(baseball);
24
25       // Display the elements of the basketball set.
26       cout << "\n\nThe following students are on the basketball team:\n";
27       displaySet(basketball);
28
29       // Display the intersection of the two sets.
30       cout << "\n\nThe following students play both sports:\n";
31       displayIntersection(baseball, basketball);
32
33       // Display the union of the two sets.
34       cout << "\n\nThe following students play either sport:\n";
35       displayUnion(baseball, basketball);
36
37       // Display the difference of baseball and basketball.
38       cout << "\n\nThe following students play baseball, "
39            << "but not basketball:\n";
40       displayDifference(baseball, basketball);
41
42       // Display the difference of basketball and baseball.
43       cout << "\n\nThe following students play basketball, "
44            << "but not baseball:\n";
45       displayDifference(basketball, baseball);
46
47       // Display the symmetric difference of the two sets.
48       cout << "\n\nThe following students play one sport, "
49            << "but not both:\n";
50       displaySymmetricDifference(basketball, baseball);
51       return 0;
52  }
53
54  // The displaySet function displays the contents of a set<string>.
55  void displaySet(set<string> s)
56  {
57     for (auto element : s)
58        cout << element << " ";
59  }
60
61  // The displayIntersection function displays the intersection
62  // of two set<string> objects.
63  void displayIntersection(set<string> set1, set<string> set2)
```

```
64  {
65      // Create a vector to hold the intersection.
66      vector<string> result(set1.size() + set2.size());
67
68      // Get the intersection of the sets.
69      auto iter = set_intersection(set1.begin(), set1.end(),
70                                   set2.begin(), set2.end(),
71                                   result.begin());
72
73      // Resize the result vector to remove unused elements.
74      result.resize(iter - result.begin());
75
76      // Display the result vector's elements
77      for (auto element : result)
78      {
79         cout << element << " ";
80      }
81  }
82
83  // The displayUnion function displays the union of two
84  // set<string> objects.
85  void displayUnion(set<string> set1, set<string> set2)
86  {
87      // Create a vector to hold the union.
88      vector<string> result(set1.size() + set2.size());
89
90      // Get the union of the sets.
91      auto iter = set_union(set1.begin(), set1.end(),
92                            set2.begin(), set2.end(),
93                            result.begin());
94
95      // Resize the result vector to remove unused elements.
96      result.resize(iter - result.begin());
97
98      // Display the result vector's elements
99      for (auto element : result)
100     {
101         cout << element << " ";
102     }
103 }
104
105 // The displayDifference function displays the difference
106 // of two set<string> objects.
107 void displayDifference(set<string> set1, set<string> set2)
108 {
109     // Create a vector to hold the union.
110     vector<string> result(set1.size() + set2.size());
111
112     // Get the difference of the sets.
113     auto iter = set_difference(set1.begin(), set1.end(),
114                                set2.begin(), set2.end(),
115                                result.begin());
116
```

*(program continues)*

**Program 17-33** *(continued)*

```
117     // Resize the result vector to remove unused elements.
118     result.resize(iter - result.begin());
119
120     // Display the result vector's elements
121     for (auto element : result)
122     {
123        cout << element << " ";
124     }
125  }
126
127  // The displaySymmetricDifference function displays the
128  // symmetric difference of two set<string> objects.
129  void displaySymmetricDifference(set<string> set1, set<string> set2)
130  {
131     // Create a vector to hold the union.
132     vector<string> result(set1.size() + set2.size());
133
134     // Get the symmetric difference of the sets.
135     auto iter = set_symmetric_difference(set1.begin(), set1.end(),
136                                          set2.begin(), set2.end(),
137                                          result.begin());
138
139     // Resize the result vector to remove unused elements.
140     result.resize(iter - result.begin());
141
142     // Display the result vector's elements
143     for (auto element : result)
144     {
145        cout << element << " ";
146     }
147  }
```

**Program Output**

```
The following students are on the baseball team:
Aida Alicia Carmen Jodi

The following students are on the basketball team:
Alicia Carmen Eva Sarah

The following students play both sports:
Alicia Carmen

The following students play either sport:
Aida Alicia Carmen Eva Jodi Sarah

The following students play baseball, but not basketball:
Aida Jodi

The following students play basketball, but not baseball:
Eva Sarah

The following students play one sport, but not both:
Aida Eva Jodi Sarah
```

### Checkpoint

17.36 When a range of elements is denoted by two iterators, to what does the first iterator point? To what does the second iterator point?

17.37 What value will be stored in `v[0]` after the following code executes?

```
vector<int> v = {8, 4, 6, 1, 9};
sort(v.begin(), v.end());
```

17.38 What must you do to a range of elements before searching it with the `binary_search()` function?

17.39 If the elements that you are sorting with the `sort()` function contain your own class objects, you must be sure that the class overloads what operator?

17.40 If the elements that you are searching with the `binary_search()` function contain your own class objects, you must be sure that the class overloads what operator?

17.41 What is a function pointer?

17.42 Assume `vect` is a vector that contains 100 `int` elements, and the following statement appears in a program:

```
for_each(vect.begin(), vect.end(), myFunction);
```

Without knowing anything else about the program, answer the following questions:

A) What is `myFunction`?
B) How many arguments does `myFunction` accept? What are the data type(s) of the argument(s)?
C) What value does `myFunction` return?
D) How many times will the statement cause `myFunction` to be called?

## 17.7 Introduction to Function Objects and Lambda Expressions

**CONCEPT: A function object is an object of a class that overloads the function call operator. Function objects behave just like functions, and can be passed as parameters to other functions. A lambda expression is a convenient way of creating a function object.**



VideoNote
**Function Objects and Lambda Expressions**

A *function object*, also known as a *functor*, is an object that acts like a function. Function objects can be called, just like regular functions. They can accept arguments, and they can return values. To create a function object, you write a class that overloads the parentheses `()` operator, which is also known as the *function call operator*. Let's look at a simple example, the `Sum` class.

### Contents of Sum.h

```
1  #ifndef SUM_H
2  #define SUM_H
3
4  class Sum
5  {
6  public:
7     int operator()(int a, int b)
8     { return a + b; }
9  };
10 #endif
```

The Sum class has one member function: the operator() function. When you write an operator() function, the function can have as many parameter variables as necessary. Notice in the Sum class, the operator() function has two int parameter variables: a and b. Also, notice in the Sum class the operator() function returns an int. The value it returns is the sum of a and b. Program 17-34 demonstrates the Sum class.

### Program 17-34

```
1  #include <iostream>
2  #include "Sum.h"
3  using namespace std;
4
5  int main()
6  {
7     // Local variables
8     int x = 10;
9     int y = 2;
10    int z = 0;
11
12    // Create a Sum object.
13    Sum sum;
14
15    // Call the sum function object.
16    z = sum(x, y);
17
18    // Display the result.
19    cout << z << endl;
20
21    return 0;
22 }
```

### Program Output

```
12
```

Let's take a closer look at Program 17-34:

- Lines 8 through 10 define three local int variables: x, y, and z.
- Line 13 creates an instance of the Sum class. The object's name is sum.
- Line 16 calls the sum object, as if it were a function. The statement passes x and y as arguments, and assigns the return value to the z variable.
- Line 19 displays the z variable.

In the previous section on STL algorithms, you saw examples of STL function templates that accept function pointers as arguments, allowing you to plug one of your own functions into the algorithm. If you prefer, you can pass a function object instead of a function pointer to any of these STL algorithms.

Let's look at an example, using the `count_if()` function. Recall the `count_if()` function iterates over a range of elements, passing each element as an argument to a function that you provide. The function you provide must accept one argument, and return either `true` or `false`. You can provide the function as either a function pointer, or a function object. The `count_if` function returns the number of elements for which *`function`* returns `true`. We will use a function object along with the `count_if()` function to count the number of even numbers that appear in a `vector`. Our function object will be an instance of the `IsEven` class, shown here:

### Contents of `IsEven.h`

```
1  #ifndef IS_EVEN_H
2  #define IS_EVEN_H
3
4  class IsEven
5  {
6  public:
7     bool operator()(int x)
8     { return x % 2 == 0; }
9  };
10 #endif
```

The `IsEven` class has one member function: the `operator()` function. The `operator()` function has one `int` parameter variable, `x`. The function returns `true` if `x` is an even number, or `false` otherwise. Program 17-35 creates an instance the `IsEven` class, and uses it as a function object with the `count_if()` function.

### Program 17-35

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  #include "IsEven.h"
5  using namespace std;
6
7  int main()
8  {
9     // Create a vector of ints.
10    vector<int> v = { 1, 2, 3, 4, 5, 6, 7, 8 };
11
12    // Create an instance of the IsEven class.
13    IsEven isNumberEven;
14
15    // Get the number of elements that even.
16    int evenNums = count_if(v.begin(), v.end(), isNumberEven);
17
```

*(program continues)*

**Program 17-35** *(continued)*

```
18      // Display the results.
19      cout << "The vector contains " << evenNums << " even numbers.\n";
20      return 0;
21  }
```

**Program Output**

```
The vector contains 4 even numbers.
```

## Constructing an Anonymous Function Object

In line 13 of Program 17-35, we created an instance of the `IsEven` class, and we named it `isNumberEven`. Then, in line 16 we passed the object to the `count_if()` function. Function objects can be called at the point of their creation, however, without ever being given a name. For example, the following code creates an `IsEven` function object and calls it with the argument 2, without ever giving the function object a name:

```
if (IsEven(2))
   cout << "The number is even.\n";
else
   cout << "The number is not even.\n";
```

Objects that are created and used without being given a name are said to be *anonymous*. Program 17-36 is similar to Program 17-35, except that an anonymous instance of the `IsEven` class is passed to the `count_if()` function.

**Program 17-36**

```
1   #include <iostream>
2   #include <vector>
3   #include <algorithm>
4   #include "IsEven.h"
5   using namespace std;
6
7   int main()
8   {
9       // Create a vector of ints.
10      vector<int> v = { 1, 2, 3, 4, 5, 6, 7, 8 };
11
12      // Get the number of elements that even.
13      int evenNums = count_if(v.begin(), v.end(), IsEven());
14
15      // Display the results.
16      cout << "The vector contains " << evenNums << " even numbers.\n";
17      return 0;
18  }
```

**Program Output**

```
The vector contains 4 even numbers.
```

## Predicate Terminology

A function or function object that returns a Boolean value is called a *predicate*. A predicate that takes only one argument is called a *unary predicate*. (An instance of the `IsEven` class, previously shown, would be a unary predicate.) A predicate that takes two arguments is called a *binary predicate*. It is important that you are familiar with this terminology because it is used in much of the available C++ documentation and literature.

## Lambda Expressions



A *lambda expression* is a compact way of creating a function object without having to write a class declaration. It is an expression that contains only the logic of the object's `operator()` member function. When the compiler encounters a lambda expression, it automatically generates a function object in memory, using the code that you provide in the lambda expression for the `operator()` member function.

Typically, a lambda expression takes the following general format:

```
[](parameter list) {  function body  }
```

The `[]` at the beginning of the expression is known as the *lambda introducer*. It marks the beginning of a lambda expression. The *`parameter list`* is a list of parameter declarations for the function object's `operator()` member function, and the *`function body`* is the code that should be the body of the object's `operator()` member function.

For example, the lambda expression for a function object that computes the sum of two integers is:

```
[](int a, int b) { return x + y; }
```

And, the lambda expression for a function object that determines whether an integer is even is:

```
[](int x) { return x % 2 == 0; }
```

As another example, the lambda expression that takes an integer as input and prints the square of that integer is written like this:

```
[](int a) { cout << a * a << " "; }
```



**NOTE:** Optionally, a *capture list* can appear inside the brackets `[]` of a lambda expression. A capture list is a list of variables in the surrounding scope of the lambda expression that can be accessed from the lambda's function body.

When you call a lambda expression, you write a list of arguments, enclosed in parentheses, right after the expression. For example, the following code snippet displays 7, which is the sum of the variables `x` and `y`:

```
int x = 2;
int y = 5;
cout << [](int a, int b) {return a + b;}(x, y) << endl;
```

The following code segment counts the even numbers in a `vector`, in the manner of Program 17-36. However, it uses a lambda expression in place of the `IsEven()` function object as the third argument to the `count_if()` function.

```
// Create a vector of ints.
vector<int> v = { 1, 2, 3, 4, 5, 6, 7, 8 };

// Get the number of elements that are even.
int evenNums = count_if(v.begin(), v.end(), [](int x) {return x % 2 == 0;});

// Display the results.
cout << "The vector contains " << evenNums << " even numbers.\n";
```

Because lambda expressions generate function objects, you can assign a lambda expression to a variable of a suitable type then call it through the variable's name. For example, the lambda expression shown in the following code gives us a function object that adds two integers. The function object is assigned to a variable named `sum`:

```
auto sum = [](int a, int b) {return a + b;};
```

Once the statement has executed, we can use the `sum` variable to call the function object, as shown here:

```
int x = 2;
int y = 5;
int z = sum(x, y);
```

After this code has executed, the variable `z` will be assigned 7. Program 17-37 shows another example. It is similar to Program 17-36, except that a lambda expression (which determines whether a value is even) is assigned to a variable named `isEven`, and that variable is passed as the third argument to the `count_if()` function.

**Program 17-37**

```
 1  #include <iostream>
 2  #include <vector>
 3  #include <algorithm>
 4  using namespace std;
 5
 6  int main()
 7  {
 8     // Create a vector of ints.
 9     vector<int> v = { 1, 2, 3, 4, 5, 6, 7, 8 };
10
11     // Use a lambda expression to create a function object.
12     auto isEven = [](int x) { return x % 2 == 0; };
13
14     // Get the number of elements that even.
15     int evenNums = count_if(v.begin(), v.end(), isEven);
16
17     // Display the results.
18     cout << "The vector contains " << evenNums << " even numbers.\n";
19     return 0;
20  }
```

**Program Output**

```
The vector contains 4 even numbers.
```

## Functional Classes in the STL

Function objects are so useful the C++ library defines a number of classes that you can instantiate to create function objects in your program. To use these classes, you must `#include` the `<functional>` header file. Table 17-15 lists those that can be used to compare objects, but you can find information on many more in various resources online.

**Table 17-15** STL Function Object Classes

| Functional Class | Description |
|---|---|
| `less<T>` | `less<T>()(T a, T b)` is true if and only if `a < b` |
| `less_equal<T>` | `less_equal()(T a, T b)` is true if an only if `a <= b` |
| `greater<T>` | `greater<T>()(T a, T b)` is true if and only if `a > b` |
| `greater_equal<T>` | `greater_equal<T>()(T a, T b)` is true if and only if `a >= b` |

For example, the STL `sort` function normally sorts the elements in a range in ascending order. This is because the `sort` function, by default, uses the `<` operator to compare elements. But what if you want to use the function to sort a range of elements in descending order? All you have to do is make the function use the `>` operator instead.

As it turns out, the `sort` function takes a comparison function as an optional third argument. The argument can be a function pointer or a function object. The function must accept two arguments (of the same type as the range elements), and must return a Boolean value. When you pass a comparison function as the third argument to the `sort` function, the `sort` function will use the comparison function instead of the `<` operator to compare range elements. To make the `sort` function use the `>` operator, you can pass an instance of the `greater<T>` class as the comparison function. Program 17-38 demonstrates how this can be done to sort a `vector` in descending order.

**Program 17-38**

```
1  #include <iostream>
2  #include <vector>
3  #include <functional>
4  #include <algorithm>
5  using namespace std;
6
7  int main()
8  {
9     // Create an unsorted vector of ints.
10    vector<int> v = {8, 1, 7, 2, 6, 3, 5, 4};
11
12    // Display the vector contents.
13    cout << "Original order:\n";
14    for (auto element : v)
15       cout << element << " ";
16
17    // Sort the vector in ascending order.
18    sort(v.begin(), v.end());
19
```

*(program continues)*

**Program 17-38** *(continued)*

```
20      // Display the vector contents again.
21      cout << "\nAscending order:\n";
22      for (auto element : v)
23         cout << element << " ";
24
25      // Sort the vector in descending order.
26      sort(v.begin(), v.end(), greater<int>());
27
28      // Display the vector contents again.
29      cout << "\nDescending order:\n";
30      for (auto element : v)
31         cout << element << " ";
32      cout << endl;
33
34      return 0;
35   }
```

**Program Output**

```
Original order:
8 1 7 2 6 3 5 4
Ascending order:
1 2 3 4 5 6 7 8
Descending order:
8 7 6 5 4 3 2 1
```



## Checkpoint

17.43 What is a function object?

17.44 Which operator must be overloaded in a class before the class can be used to create function objects?

17.45 What is an anonymous function object?

17.46 What is a predicate?

17.47 What is a unary predicate?

17.48 What is a binary predicate?

17.49 What is a lambda expression?

# Review Questions and Exercises

## Short Answer

1. What two emplacement member functions are provided by the `vector` class? How are these member functions different from the `insert()` and `push_back()` member functions?
2. What is the difference between the `vector` class's `size()` member function and the `capacity()` member function?

3. If you want to store objects of a class that you have written as values in a `map`, what requirement must the class meet?
4. If you want to store objects of a class that you have written as keys in a `map`, what requirement must the class meet?
5. What is the difference between a `map` and an `unordered_map`?
6. What is the difference between a `map` and a `multimap`?
7. What happens if you use the `insert()` member function to insert a value into a `set`, and that value already exists in the `set`?
8. If you want to store objects of a class that you have written as keys in a `set`, what requirement must the class meet?
9. What is the difference between a `set` and a `multiset`?
10. How does the behavior of the `count()` member function differ between the `set` class and the `multiset` class?
11. How does the behavior of the `equal_range()` member function differ between the `set` class and the `multiset` class?
12. What are two differences between the `set` and `unordered_set` classes?
13. When using one of the STL algorithm function templates, you typically work with a range of elements that are denoted by two iterators, to what does the first iterator point? To what does the second iterator point?
14. You have written a class, and you plan to store objects of that class in a `vector`. If you plan to use the `sort()` and/or `binary_search()` functions on the `vector`'s elements, what operator must the class overload?
15. What is a function object?
16. If you want to create function objects from a class, what must the class overload?
17. What is an anonymous function object?
18. What is a lambda expression?

## Fill-in-the-Blank

19. There are two types of container classes in the STL: _____ and _____.
20. A(n) _____ container organizes data in a sequential fashion similar to an array.
21. A container _____ class is not itself a container, but a class that adapts a container to a specific use.
22. A(n) _____ container stores its data in a nonsequential way that makes it faster to locate elements.
23. _____ are pointer-like objects used to access data stored in a container.
24. Each element that is stored in a `map` has two parts: a _____ and a _____.
25. _____ is a map container that allows duplicate keys.
26. The _____ class is an associative container that stores a collection of unique, sorted values.
27. The _____ header file defines several function templates that implement useful algorithms.

28. A _____ is a pointer to a function's executable code.
29. A _____ object is an object that can be called, like a function.
30. A _____ is a function or function object that returns a Boolean value.
31. A _____ is a predicate that takes one argument.
32. A _____ is a predicate that takes two arguments.
33. A _____ is a compact way of creating a function object without having to write a class declaration.

## True or False

34. T F The `array` class is a fixed-size container.
35. T F The `vector` class is a fixed-size container.
36. T F You use the `*` operator to dereference an iterator.
37. T F You can use the `++` operator to increment an iterator.
38. T F A container's `end()` member function returns an iterator pointing to the last element in the container.
39. T F A container's `rbegin()` member function returns a reverse iterator pointing to the first element in a container.
40. T F You do not have to declare the size of a `vector` when you define it.
41. T F A `vector` uses an array internally to store its elements.
42. T F A `map` is a sequence container.
43. T F You can store duplicate keys in a map container.
44. T F The `multimap` class's `erase()` member function erases only one element at a time. If you want to erase multiple elements that all have the same key, you will have to call the `erase()` member function multiple times.
45. T F All the elements in a `set` must be unique.
46. T F The elements in a `set` are sorted in ascending order.
47. T F If the same value appears more than once in the initialization list of a `set` definition, an exception will occur at runtime.
48. T F The `unordered_set` container has better performance than the `set` container.
49. T F If two iterators denote a range of elements that will be processed by an STL algorithm function, the element pointed to by the second iterator is not included in the range.
50. T F You must sort a range of elements before searching it with the `binary_search()` function.
51. T F Any class that will be used to create function objects must overload the `operator[]` member function.
52. T F Writing a lambda expression usually requires more code than writing a function object's class declaration and then instantiating the class.
53. T F You can assign a lambda expression to a variable, and then use the variable to call the lambda expression's function object.

## Algorithm Workbench

54. Write a statement that defines an `array` object named `numbers` that can hold ten elements of the `double` data type.
55. Write a statement that defines an iterator that can be used with the `array` object that you defined in question 54. The iterator's name should be `it`.
56. Write a statement that defines a `vector` named `numbers` that can hold elements of the `int` data type. Initialize the `vector` with the values 10, 20, 30, 40, and 50.
57. The following statement defines a `vector` of `int`s named `v`. Write a statement that defines another `vector`, named `v2`, which is a copy of `v`.

```
vector<int> v = {1, 2, 3, 4, 5};
```

58. Write a range-based `for` loop that displays all of the elements of the `vector` that you defined in question 56.
59. Write a `for` loop that uses an iterator to display all of the elements of the `vector` that you defined in question 56.
60. The following code defines a `vector` and an iterator. Rewrite the statement that defines the iterator so it uses the `auto` key word.

```
vector<string> strv = {"one", "two", "three"};
vector<string>::iterator it = strv.begin();
```

61. The following statement defines a `vector` of `int`s named `v`. Write a statement that defines a `const_iterator` named `cit`, and initializes it to point to the first element in `v`.

```
vector<int> v = {1, 2, 3, 4, 5};
```

62. The following statement defines a `vector` of `int`s named `v`. The `vector`'s initial contents are: 1, 2, 3, 4, 5. Write code to insert the value 999 into the `vector`, so its contents are: 1, 2, 999, 3, 4, 5.

```
vector<int> v = {1, 2, 3, 4, 5};
```

63. Write a statement that defines a `map` named `food`. The keys should be `string`s, and the values should be `int`s.
64. Suppose a program defines a `map` as follows:

```
map<int, string> customers;
```

Write statements that use the `map` class's `emplace` member function to insert the following elements:

```
9001, "Jen Williams"
9002, "Frank Smith"
9003, "Geri Rose"
```

65. Look at the following `vector` definition:

```
vector<int> v = {7, 2, 1, 6, 4, 3, 5};
```

Write code that displays a message indicating whether the value 6 is found in the `vector`. Use the STL `binary_search()` algorithm to search the `vector` for the value 6.

66. Write a declaration for a class named `Display`. The class should be written in such a way that it can be used to create a function object. A function object created from the class should accept an `int` argument, and display that argument on the screen.

67. Write code that does the following:
    - Uses a lambda expression that accepts two `int` arguments, `a` and `b`, and returns the value of `a * b`.
    - Assigns the lambda expression to a variable named `multiply`.
    - Uses the `multiply` variable to call the lambda expression, passing the values 2 and 10 as arguments. The result should be assigned to an `int` variable named `product`.

### Find the Errors

Each of the following code snippets has errors. Find as many as you can.

68.
```
// This code has an error.
array<int, 5> a;
a[5] = 99;
```

69.
```
// This code has an error.
vector<string> strv = {"one", "two", "three"};
vector<string>::iterator it = strv.cbegin();
```

70.
```
// This code has an error.
vector<int> numbers(10);
for (int index = 0; index < numbers.length(); index++)
   numbers[index] = index;
```

71.
```
// This code has an error.
vector<int> numbers = {1, 2, 3};
numbers.insert(99);
```

72.
```
// This code has an error.
map<string, string> contacts;
contacts.insert("Beth Young", "555-1212);
```

73.
```
// This code has an error.
multimap<string, string> phonebook;
phonebook["Megan"] = "555-1212";
```

74.
```
// This code has an error.
vector<int> v = {6, 5, 4, 2, 3, 1};
sort(v);
if (binary_search(v, 1))
   cout << "The value 1 is found in the vector.\n";
else
   cout << "The value 1 is NOT found in the vector.\n";
```

75.
```
// This code has an error.
auto sum = ()[int a, int b] { return a + b; };
```

## Programming Challenges

1. **Unique Words**

   Write a program that opens a specified text file then displays a list of all the unique words found in the file.

   *Hint: Store each word as an element of a set.*


VideoNote
**The Course Information Problem**

**2. Course Information**

Write a program that creates a `map` containing course numbers and the room numbers of the rooms where the courses meet. The `map` should have the following key–value pairs:

| Course Number (Key) | Room Number (Value) |
|---|---|
| CS101 | 3004 |
| CS102 | 4501 |
| CS103 | 6755 |
| NT110 | 1244 |
| CM241 | 1411 |

The program should also create a `map` containing course numbers and the names of the instructors that teach each course. The `map` should have the following key–value pairs:

| Course Number (Key) | Instructor (Value) |
|---|---|
| CS101 | Haynes |
| CS102 | Alvarado |
| CS103 | Rich |
| NT110 | Burke |
| CM241 | Lee |

The program should also create a `map` containing course numbers and the meeting times of each course. The `map` should have the following key–value pairs:

| Course Number (Key) | Meeting Time (Value) |
|---|---|
| CS101 | 8:00 a.m. |
| CS102 | 9:00 a.m. |
| CS103 | 10:00 a.m. |
| NT110 | 11:00 a.m. |
| CM241 | 1:00 p.m. |

The program should let the user enter a course number, then it should display the course's room number, instructor, and meeting time.

**3. Tokenized Strings to JSON**

The JSON (JavaScript Object Notation) format is becoming more and more popular. The format is quite flexible, but in its most basic format everything is expressed as key–value pairs in the format "key": "value", with pairs delimited by a comma, like {"book": "C++", "author": "Gaddis", "chapters": "21"}. Write a program that first creates a map from the values in the file challenge3.txt and prints out the (simplified) JSON format described above.

**4. File Encryption and Decryption**

Write a program that uses a `map` to assign "codes" to each letter of the alphabet. For example:

```
map<char, char> codes =
  { {'A', '%'}, {'a', '9'}, {'B', '@'}, {'b', '#'}, etc ...};
```

Using this example, the letter 'A' would be assigned the symbol %, the letter 'a' would be assigned the number 9, the letter 'B' would be assigned the symbol @, and so forth. The program should open a specified text file, read its contents, then use the `map` to write an encrypted version of the file's contents to a second file. Each character in the second file should contain the code for the corresponding character in the first file. Write a second program that opens an encrypted file and displays its decrypted contents on the screen.

5. **Text File Analysis**

   Write a program that reads the contents of two text files and compares them in the following ways:

   - It should display a list of all the unique words contained in both files.
   - It should display a list of the words that appears in both files.
   - It should display a list of the words that appears in the first file, but not the second.
   - It should display a list of the words that appears in the second file, but not the first.
   - It should display a list of the words that appears in either the first or second file, but not in both.

   *Hint: Use set operations to perform these analyses. Also, see Chapter 10 for a discussion of string tokenizing.*

6. **Word Frequency**

   Write a program that reads the contents of a text file. The program should create a `map` in which the keys are the individual words found in the file and the values are the number of times each word appears. For example, if the word "the" appears 128 times, the `map` would contain an element with "the" as the key and 128 as the value. The program should either display the frequency of each word or create a second file containing a list of each word and its frequency.

   *Hint: See Chapter 10 for a discussion of string tokenizing.*

7. **Word Index**

   Write a program that reads the contents of a text file. The program should create a `map` in which the key–value pairs are described as follows:

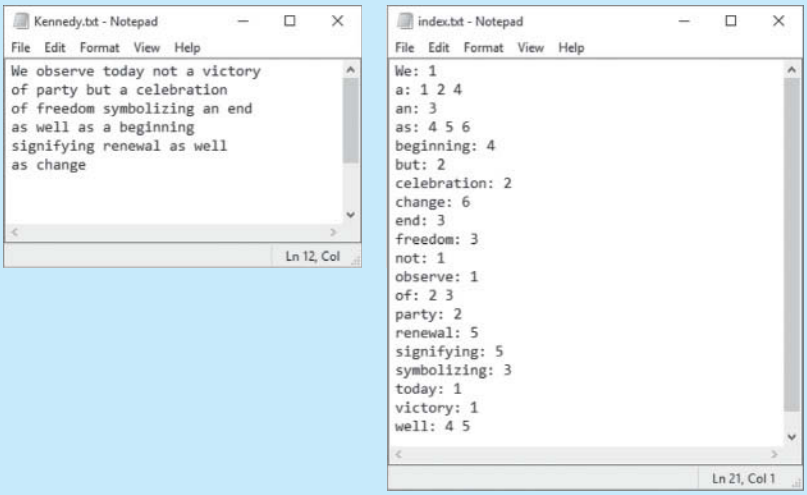
   - **Key**—The keys are the individual words found in the file.
   - **Values**—Each value is a `set` that contains the line numbers in the file where the word (the key) is found.

   For example, suppose the word "robot" is found in lines 7, 18, 94, and 138. The `map` would contain an element in which the key was the string "robot", and the value was a `set` containing the numbers 7, 18, 94, and 138.

   Once the `map` is built, the program should create another text file, known as a word index, listing the contents of the `map`. The word index file should contain an alphabetical listing of the words that are stored as keys in the map, along with the line numbers where the words appears in the original file. Figure 17-9 shows an example of an original text file (Kennedy.txt) and its index file (index.txt).

   *Hint: See Chapter 10 for a discussion of string tokenizing.*

**Figure 17-9** Example original file and index file




8. **Prime Number Generation**

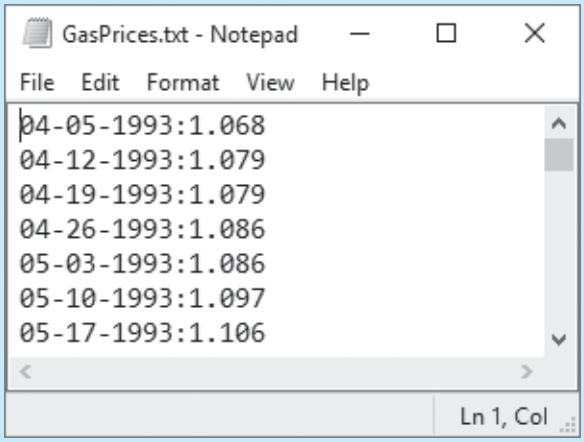
   A positive integer greater than 1 is said to be *prime* if it has no divisors other than 1 and itself. A positive integer greater than 1 is *composite* if it is not prime. Write a program that asks the user to enter an integer greater than 1, then displays all of the prime numbers that are less than or equal to the number entered. The program should work as follows:

   - Once the user has entered a number, the program should populate a `vector` with all of the integers from 2, up through the value entered.
   - The program should then use the STL's `for_each` function to step through the `vector`. The `for_each` function should pass each element to a function object that displays the element if it is a prime number.

9. **Gas Prices**

   In the student sample program files for this chapter, you will find a text file named GasPrices.txt. The file contains the weekly average prices for a gallon of gas in the U.S., beginning on April 5, 1993, and ending on August 26, 2013. Figure 17-10 shows an example of the first few lines of the file's contents.

**Figure 17-10** The GasPrices.txt file



Each line in the file contains the average price for a gallon of gas on a specific date. Each line is formatted in the following way:

```
MM-DD-YYYY:Price
```

*MM* is the two-digit month, *DD* is the two-digit day, and *YYYY* is the four-digit year. *Price* is the average price per gallon of gas on the specified date.

For this assignment, you are to write one or more programs that read the contents of the file and perform the following calculations:

**Average Price Per Year:** Calculate the average price of gas per year, for each year in the file. (The file's data starts in April of 1993, and it ends in August 2013. Use the data that is present for the years 1993 and 2013.)

**Average Price Per Month:** Calculate the average price for each month in the file.

**Highest and Lowest Prices Per Year:** For each year in the file, determine the date and amount for the lowest price, and the highest price.

**List of Prices, Lowest to Highest:** Generate a text file that lists the dates and prices, sorted from the lowest price to the highest.

**List of Prices, Highest to lowest:** Generate a text file that lists the dates and prices, sorted from the highest price to the lowest.

You can write one program to perform all of these calculations, or you can write different programs, one for each calculation. Regardless of the approach that you take, you should read the contents of the GasPrices.txt file, and extract its data into one or more STL containers appropriate for your algorithm.

*Hint: See Chapter 10 for a discussion of string tokenizing.*